UNIVERSITY OF MILANO-BICOCCA

Department of Computer Science, Systems and Communication

Master's Degree Programme in **Computer Science**

Innovative Management of Service Requests: From a Distributed Monolith to a Large-Scale Microservices Architecture, with RAG-Enhanced Reccomendation System

Advisor: Prof. Leonardo Mariani

Master's Thesis by:
Nicolas Guarini
Student ID: 918670

Academic Year 2024–2025

“What I cannot create, I do not understand”

- Richard Feynman

Contents

Introduction

The technological evolution of recent decades has profoundly transformed the business landscape, making IT solutions a crucial factor for the success of enterprises. In this context, Elmec Informatica S.p.A. stands out as a significant player in the Information Technology sector in Italy. Founded in 1971 and headquartered in Varese, Elmec combines extensive industry experience with a strong focus on innovation, offering services and solutions that cover a wide range of business needs, offering advanced IT solutions for medium and large enterprises.

The company manages four datacenters (including one Tier IV certified), provides Hybrid IT services, digital workspace and Device-as-a-Service solutions, and is a key partner for cybersecurity and regulatory compliance. With over 450 certified technicians, Elmec integrates cloud technologies (in collaboration with AWS and GAIA-X), promotes environmental sustainability, and stands out for its innovation through its Technological Campus and ongoing investments in professional training.

The primary context of this internship revolves around the redesign and enhancement of the existing *Service Request Portal (SRP)*, which is essential for managing client service requests efficiently.

A Service Requests is a formal request submitted by a customer to obtain specific services or actions within their IT infrastructure. These requests can encompass a wide range of activities, from creating new user accounts in the Active Directory system to managing user permissions, resetting passwords, and deploying software updates.

Various challenges have been identified within the current architecture, particularly concerning the dynamic forms used by customers for submitting the service requests, the management of customizable fields, and the integration with external data sources.

The main objectives of this internship are:

- **Analysis of the Existing System**: Conduct a comprehensive analysis of the current state of the SRP system, identifying weaknesses and inefficiencies that hinder its performance and user experience;

- **Architectural redesign**: Propose and implement a complete architectural redesign of the system;

- **Modular and Scalable Solution**: Design a modular and scalable solution that can accomodate various client-specific configurations;
- **Migrations**: Plan and execute a full migration of databases and other company services/platforms that use SRP.
- **Fine-tuned LLMs integration**: Explore opportunities for integrating specifically fine-tuned LLM systems into the SR workflow.

Chapter 1

Overview of the existing System

The internship project was not focused on creating an entirely new system from scratch, but rather on a deep redesign and restructuring of an existing system developed many years ago using legacy technologies and affected by several issues and critical limitations.

Before discussing the architectural, technological, and implementation choices that were made, it is necessary to carry out a thorough analysis of the current system, in order to understand its behavior, operational flows, technologies used, the main issues to be addressed, how to resolve them, how to prevent them from reoccurring in the future, and how to ensure that the new system is scalable and maintainable.

1.1 Service Requests Workflow

The lifecycle of a Service Request within the current system is orchestrated through a series of defined states and automated processes, involving interactions between various components and human operators. The flow is designed to manage SRs from initiation to completion, handling approvals, execution, and potential errors [26].

In Figure 1.1 [26], a simple and high-level flowchart is shown that illustrates the main flows and scenarios.

1.1.1 SR Initiation and Initial State Management

A Service Request can be initiated through two primary channels [26]:

- **Via MyElmec Portal**: Clients can submit an SR by completing a dedicated form from the MyElmec portal. Upon saving the form, a ticket is created in the HelpdeskAdvanced (HDA) system with a `QUALIFIED` status, and a corresponding Service Request is generated in SRP with a `NEW` status. These two entities are then linked via the HDA Ticket ID;

- **Via HDA Ticket by Elmec Operator**: Operators have the ability to associate a catalog SR directly with a ticket they are managing within the HDA interface. When the ticket is saved in a `DELEGATED` status, the SR

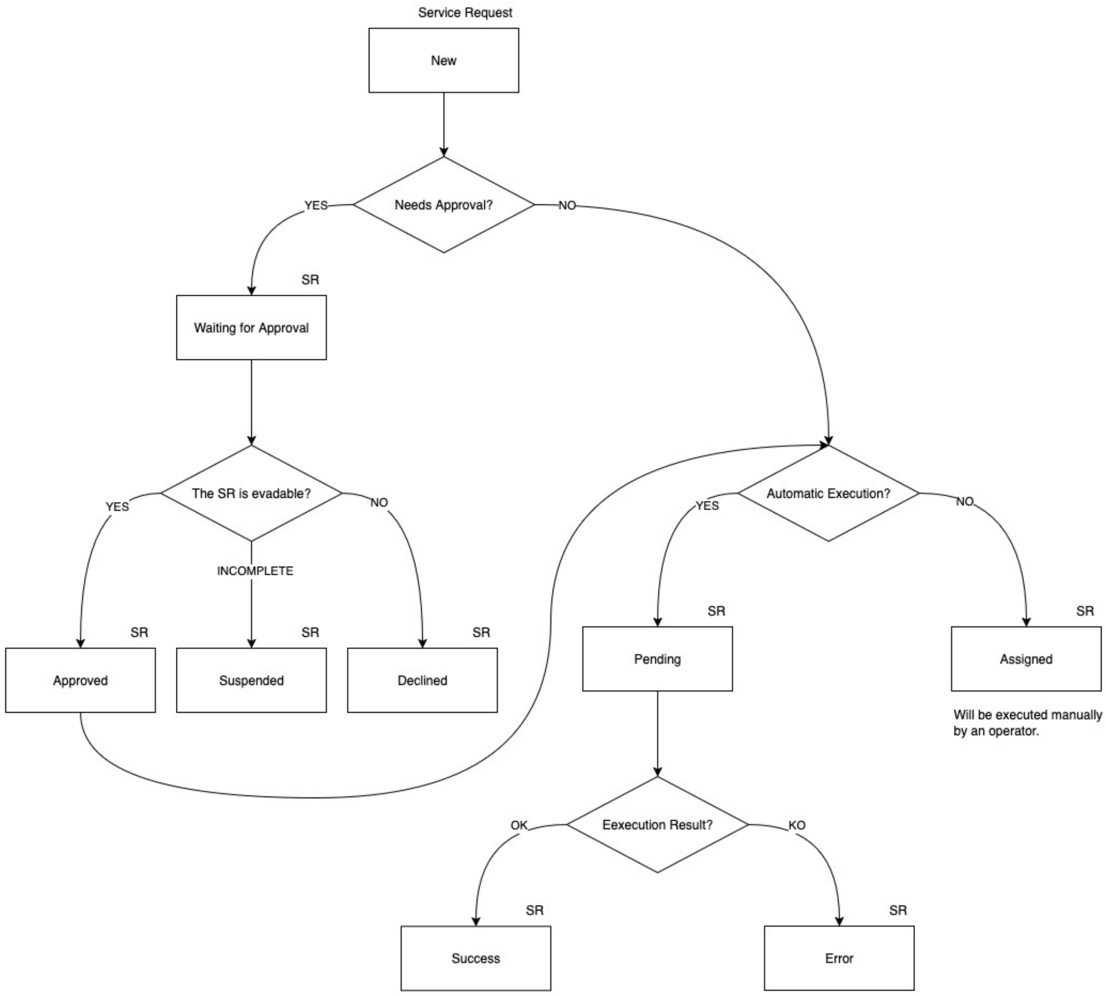

Figure 1.1: High-Level Service Request Lifecycle

is created in SRP with a `NEW` status, linked to the ticket, and immediately triggers the SR workflow within SRP.l

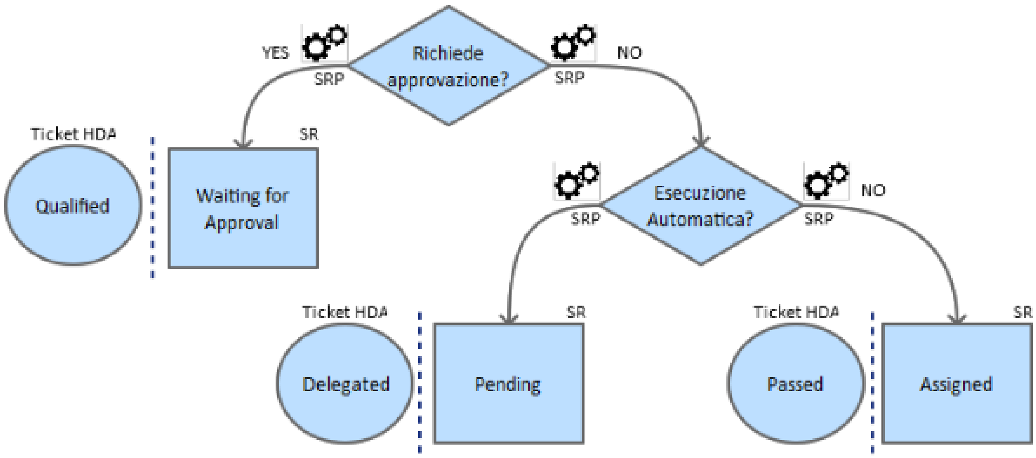

Figure 1.2: AS-IS New Service Request Workflow

As shown in Figure 1.2, following the Service Request creation, the system evaluates whether the Service Request requires approval by the operators. The logic for setting the initial SR and HDA ticket states is as follows [26]:

- **If approval is required**: the Service Request is set to `WAITING FOR APPROVAL` status, and the corresponding HDA ticket is set to `QUALIFIED`. A communication is added to the ticket, indicating that the SR needs validation from an operator before execution;
- **If no approval is required and it's linked to an automatism**: the Service Request enters a `PENDING` state, and the HDA ticket is set to `DELEGATED`;
- **If no approval is required and it's not linked to an automatism**: the Service Request is `ASSIGNED`, and the HDA ticket is set to `PASSED`. A communication is added to the ticket, noting that the SR requires manual processing by an operator.

1.1.2 Approval and Execution Workflows

As shown in Figure 1.3, for SRs requiring approval (`WAITING FOR APPROVAL` state with an HDA `QUALIFIED` ticket), an operator's intervention is necessary to validate the request. The operator has three possible actions [26]:

- **Cancel the SR**: the HDA ticket is set to `CLOSED ABORTED`, the SR status becomes `DECLINED`, and a notification email is sent to the requesting client;
- **Request client modification**: the HDA ticket is set to `WAITING USER`, the SR status becomes `SUSPENDED`, and a notification email is sent to

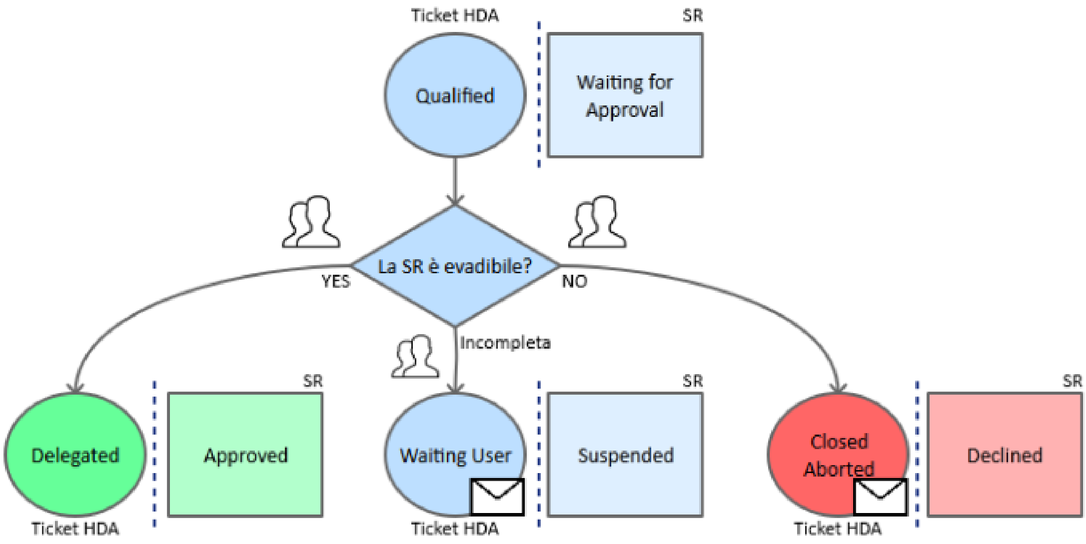

Figure 1.3: AS-IS Approval Service Request Workflow

the client. The workflow pauses until the client modifies the SR via Elmec.com;

- **Approve the SR**: the HDA ticket is set to `DELEGATED`, and the SR status becomes `APPROVED`.

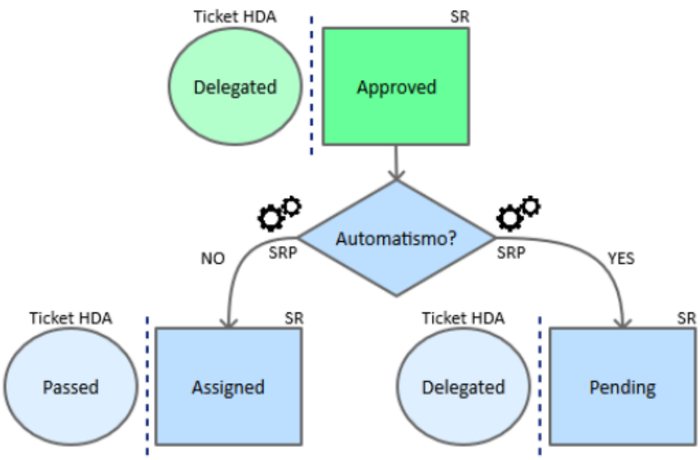

Figure 1.4: AS-IS Approved Service Request Workflow

As shown in Figure 1.4, once a SR is approved, SRP manages its subsequent flow based on whether it is linked to an automatism [26]:

- **If the SR is linked to an automatism**: the SR transitions to a `PENDING` state, while the HDA ticket remains `DELEGATED`. This implies it's awaiting automated execution;

- **If the SR is not linked to an automatism**: the SR is immediately set to `ASSIGNED`, and the HDA ticket is moved to `PASSED`. A note is added to the ticket indicating that the SR requires manual processing.

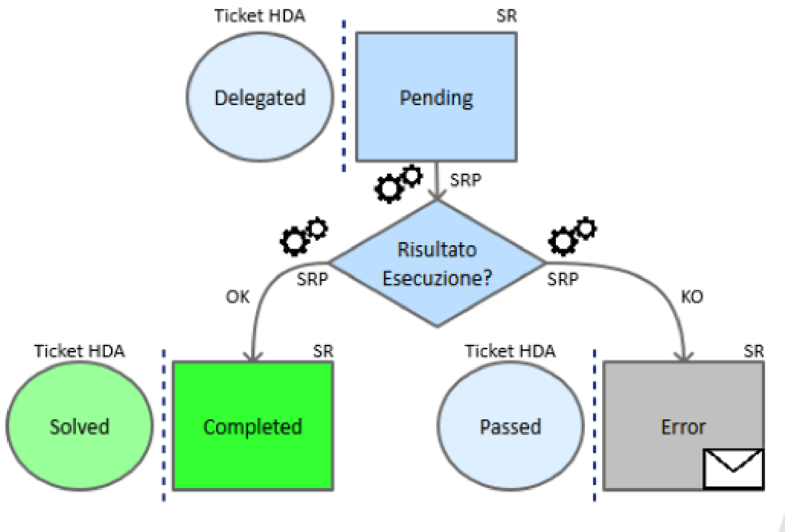

Figure 1.5: AS-IS Pending Service Request Workflow

As shown in Figure 1.5, an SR in `PENDING` state is awaiting the execution of its associated automatism. The outcome of this automated execution determines the next state [34]:

- **Successful execution**: the SR is set to `COMPLETED`, and the corresponding HDA ticket is set to `SOLVED`;
- **Unsuccessful execution**: the SR transitions to an `ERROR` state, and the HDA ticket is set to `PASSED`. Communication regarding the error is added to the ticket for visibility within HDA.

The execution of Service Requests themselves can involve the SRP Agent on the client's Domain Controller for operations requiring client-side interaction (e.g., Active Directory or Exchange environment changes). The SRP Agent periodically checks `ELMEC-SRP01` for pending SRs. If found, it retrieves the SR details, loads the relevant PowerShell script, and executes it. The agent then informs ELMEC-SRP01 that the SR is running [34]. Monitoring is performed by a scheduled `READLOG` task on the client side, which checks the PowerShell script's log file every minute. A "KEY" in the log indicates completion, prompting the SRP Agent to inform ELMEC-SRP01 to set the SR to COMPLETED, triggering an HDA status update. An "ERROR" key indicates failure, leading to the log being sent to ELMEC-SRP01 and an error signal. A timeout mechanism also signals an error if completion is not detected within a specified duration [34].

A constraint that must be respected is that the high-level flow (state transitions and mapping between SR states and HDA ticket states) must not be modified, as it represents an established and approved process that also encompasses organizational and accountability aspects among various Elmec departments

[27]: those who configure and manage the catalog and related Service Requests (Managed Services (MxS) team), those who develop and maintain the product (Innovation team), and those responsible for the ticket management system (HDA team). The technological and implementation aspects, on the other hand, can and should be redesigned and improved [27], as will be explored in the following chapters.

1.2 Current System Architecture

The *SRP* system is fundamentally divided into two primary segments:

- Company Network;
- Client Network.

These segments interact and communicate exclusively through a *DMI Console* [28], which acts as a unidirectional bridge, transferring requests from the client network to the internal company network [28] [13].

The overall architecture is depicted in the diagram in figure 1.6 [28].

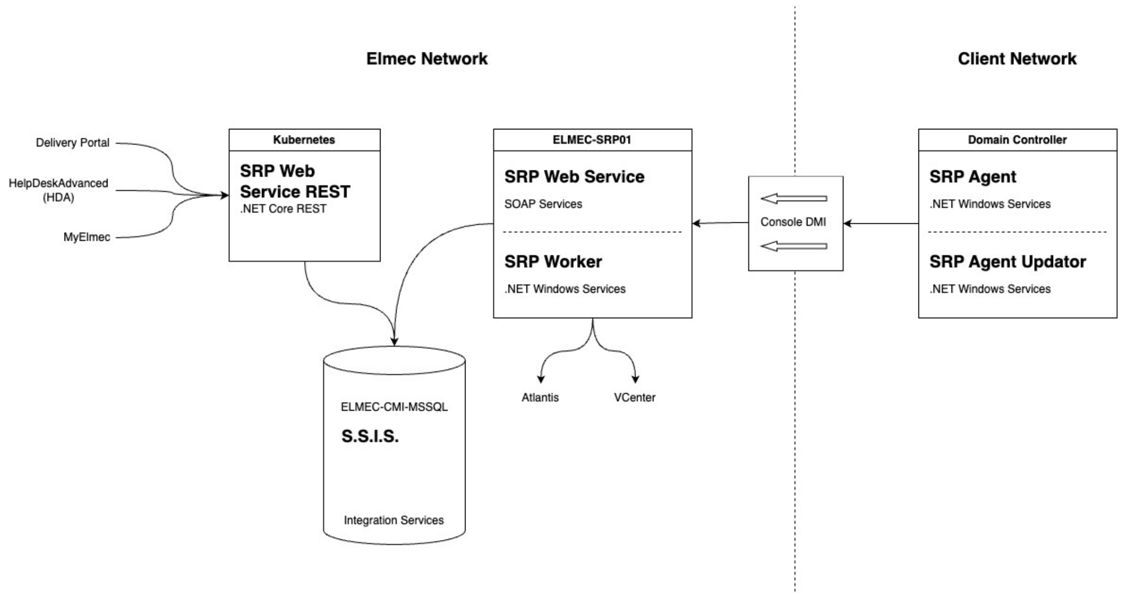

Figure 1.6: AS-IS System Architecture

The system's operational flow involves several key components across both networks, orchestrating the processing of service requests.

1.2.1 Main Functionalities and Components

The current *SRP* system relies on a suite of interconnected components, each serving a specific role in the lifecycle of a service request.

Elmec's Network Components:

- **SRP Web Sevice REST** [33]: This component, deployed on a Kubernetes cluster, exposes .NET Core RESTful services [28]. It serves as the primary interface for external systems such as:

  - **Delivery Portal**: An asset management system used to manage Service Request catalog configuration and Elmec's internal and client servers;

  - **HDA (HelpDeskAdvanced)**: A ticket management system. Each service request is intrinsically linked to an HDA ticket via an ID, and HDA is responsible for initiating and associating service requests with existing tickets.

  - **MyElmec**: A client-facing portal enabling customers to create, manage, and monitor the various services provided by Elmec (e.g., servers, management software, cloud solutions).

  The *SRP Web Service REST* communicates with the *ELMEC-CMI-MSSQL* server.

- **ELMEC-CMI-MSSQL**: This server hosts the central SQL Server Database for the SRP system. It is the persistent storage layer for all service request data and related information [28].

- **S.S.I.S. (SQL Server Integration Services)**: Running on the ELMEC-CMI-MSSQL server, SSIS packages are crucial for data transformation and import/export operations. They facilitate the transfer of data, such as master data (e.g., Active Directory users, Organizational Units, groups) sent by SRP Agents, into the database. Each SSIS package is designed to transform the received information for database insertion or vice versa [28].

- **ELMEC-SRP01**: This Windows Server functions as the core engine for service requests within the Elmec network. It houses all the PowerShell scripts associated with individual service requests, which are maintained by the Platform team [28]. Key components residing on ELMEC-SRP01 include:

  - **SRP WebServiceSoap**: This component provides SOAP services primarily for communication with the SRP Agent located on the client side. It interacts with the database to retrieve necessary information before exposing it to the agents for service request execution [32].

  - **SRP InternalWorker**: A Windows service that operates similarly to the SRP Agent but executes entirely within the Elmec network. This is utilized for service requests that do not require interaction with the client's network or Active Directory (e.g., "no patch" service requests). The InternalWorker queries the database every 10 seconds for "internal worker" type jobs (like removing a server from a patching session, or resetting a managed Virtual Machine), connects to the SOAP service, retrieves the Service Request ID and its fields, and initiates job execution. It also monitors logs every 30 seconds to determine the job's status [31].

The only component in the client's network is called **Domain Controller**, which represents the client's Active Directory domain controller, where the communication tools with Elmec are installed. It hosts:

- **SRP Agent (.NET Windows Service)**: Typically, one agent is configured per client domain, specifically for Active Directory (AD) and Exchange-related service requests. This agent performs two main functions [34]:
  - Every 8 hours, it reads master data (AD users, OUs, groups) from the domain controller and sends it to ELMEC-SRP01 for storage via SSIS.
  - Every 5 minutes, it queries ELMEC-SRP01 for pending Service Requests. Upon finding any, it requests detailed information, loads the corresponding PowerShell script, and executes it. After initiating the script, the agent informs ELMEC-SRP01 that the SR is "running" and proceeds to the next SR.
- **SRP Agent Updator (.NET Windows Service)**: This service continuously monitors the SRP Agent folder for a file with a .new extension. If detected, it stops the SRP Agent, updates it, and restarts it

1.2.2 Interactions and Communication between Customers and Company networks

The Communication between the company network and the clients' networks is a critical aspect of the SRP system's operations and is handled through the **DMI Console**.

The DMI console serves as a key infrastructural component, primarily acting as a communication bridge between the customer's network and Elmec's internal network. Specifically, for the SRP system, the DMI console is responsible for transferring service requests (SR) from the customer's network to Elmec's internal network, although it does not handle traffic in the opposite direction [13].

Operation of the DMI Console

The connectivity of the DMI appliances to Elmec is established through multiple OpenVPN connections, initiated by the appliance towards the Brunello 4 and Mendrisio Data Centers (for Swiss customers). The DMI consoles are not directly exposed to the Internet and are accessible only by authorized personnel. Hardware requirements for a DMI node include 4 GB of RAM, 4 cores, and 50 GB of disk space [13].

Following recent developments in the DMI, the new consoles use K3S for container management. K3S is a lightweight, certified Kubernetes distribution designed for edge, IoT, and CI/CD environments. It is characterized by being a single binary that reduces external dependencies and uses SQLite as the default database, greatly simplifying installation and management. It already includes essential components such as *containerd* [18].

All consoles are centrally managed via Rancher, and the deployment of the application stack is delegated to ArgoCD, which ensures that container versions are constantly updated. For security reasons, the console in the customer network by default exposes only SSH, while services such as SRP are selectively enabled and only when necessary. All application data present on the consoles is stored on a LUKS (Linux Unified Key Setup) encrypted volume[1], which can only be unlocked if the VPN tunnel is established. The operating system update is performed through a standard Elmec system, called PMD (Patching Management Dashboard) [13].

1.3 Problems and core issues of the current system

The analysis of the architecture and workflow of the current SRP system has revealed a number of critical issues that compromise its scalability, maintainability, and reliability. These problems are deeply rooted in the technological and design choices made years ago and are evident across several key areas of the system, from catalog management to script execution and the import of master data from Active Directory.

[1]A platform-independent hard disk encryption method that can be used to encrypt disks across a wide variety of tools. This not only ensures full compatibility and interoperability between different software but also guarantees that password management occurs in a secure and documented manner [9].

1.3.1 Architectural and Technological Challenges

The SRP system is characterized by a fragmented architectural structure and complex interdependencies between its main components, which leads to several critical issues.

Monolithic Frontend Component

The user interface, responsible for navigating the Service Request catalog and completing the associated forms, is an excessively large and complex Vue.js frontend component (nearly 11,000 lines). This architecture makes modifications, extensions, and client-specific customizations exceptionally challenging [21]. The logic for managing dynamic fields, their interdependencies, and the rules for compilation and validation are all handled at the frontend level through a long series of hard-coded conditional statements, such as the following:

```
<div v-if="field.desc == 'Domain'">
    ...
</div>
```

This, in addition to mixing business logic with presentation, represents a significant challenge for catalog configuration, as there is no centralized way to manage the fields and their relationships. Every change, therefore, requires a direct intervention on the code, making the system inflexible and difficult to maintain, especially in a context where the operator modifying the catalog is not a programmer and does not know how the frontend code is structured. For instance, if an operator wanted to add a new "Domain" field to a Service Request, this field would need to have the exact description "Domain" and type "domain"; otherwise, the Vue.js component would not render it correctly.

These dynamics make the system fragile and susceptible to frequent errors that waste time and resources for both clients and operators [21].

Overly Generic Backend REST Service

The only information about the fields returned by the backend REST service (`SRPWebSeviceREST`), besides the field name and its mandatoriness, is its *type*. It is therefore easy to imagine how this information, which should only be an indication of the component that needs to be rendered (e.g., textbox, select, radio button, etc.), can become an identifier for something much broader.

For example, a field of type `UPN` is a composite field derived from the value of another field, '`Display Name`', concatenated with an '`@`' symbol, and suffixed with the value from a select field whose options are the client's domains, provided by an external REST service (e.g., "`n.guarini@elmec.it`", where "`n.guarini`" is the value of the "`Display Name`" field, and "`elmec.it`" is the value of the "`Domain`" select field).

Despite this complexity, the REST service returns only a JSON document of this type:

```json
{
    "idField": 1603,
    "descField": "UPN",
    "type": "upn",
    "mandatory": true
}
```

thus delegating to the frontend the responsibility of interpreting the type and rendering the field correctly.

Dependency on the SRP Agent

The import of data from clients' Active Directories and the actual execution of Service Requests are handled by an agent (`SRPWindowsService`) installed directly on the clients' Domain Controllers. This approach presents several challenges:

- **Security and Permissions**: The agent often requires credentials with elevated privileges (e.g., `domain-admin`), which clients are increasingly reluctant to grant.
- **Reliability and Control**: The configurations on the agent and the Domain Controller are complex and difficult to manage centrally, which can cause execution errors and data synchronization issues.
- **Scalability**: The need to install, configure, and maintain an agent for each client domain represents a significant operational effort. Alternatives such as more modern provisioning systems will be evaluated.
- **Obsolete Communication Patterns**: The unidirectional communication through the *DMI Console* creates a bottleneck and complicates the management of asynchronous operations. Communication between components in the client's network and the company network is based on polling (e.g., the SRP Agent queries the server every 5 minutes), an approach that introduces latency and inefficiencies.

1.3.2 Code and Data Management Issues

Code and data persistence management is another critical aspect of the system, characterized by a lack of versioning and modern debugging tools.

Unversioned and Undebuggable Stored Procedures

The intensive use of Stored Procedures[2] in the MSSQL database represents one of the major weaknesses. These procedures contain fundamental business logic and are:

- **Unversioned**: There is no centralized version control system for Stored Procedures. Changes are made directly on the database, making it impossible to track changes, revert to previous versions, or test modifications in separate environments.

- **Difficult to Debug**: Debugging Stored Procedures is a complex and cumbersome process, often requiring direct access and specific SQL server tools, with the risk of affecting the production environment.

Inaccessible and Unversioned SSIS Jobs

SSIS packages[3] are essential for data import and export operations (e.g., AD master data). However, these jobs reside physically on the `ELMEC-CMI-MSSQL` server, and the only way to access and modify them is via a remote desktop. This approach prevents versioning and collaboration, limits accessibility by making maintenance and debugging an onerous and risky activity, and creates a *single-point-of-failure* and a strong coupling between the import/export business logic and the infrastructure.

1.3.3 Database Schema and Data Integrity Issues

The analysis of the database schema, represented in figures 1.7, has revealed a number of significant issues that affect the quality and integrity of the data managed by the SRP system.

Lack of Data Integrity and Consistency

One of the most significant problems concerns the lack of explicit referential integrity constraints. In the schema in figure 1.7, although the relationships between tables have been inferred manually, they are not defined at the database level through the use of foreign keys. This absence prevents the

[2]Stored procedures are SQL code routines, or sets of instructions, that are pre-compiled and stored within a relational database. They act as logical units of execution, encapsulating business logic directly at the database level [36].

[3]SQL Server Integration Services (SSIS) is a platform for building data integration and workflow solutions. It is a key component of Microsoft SQL Server, primarily used for Extract, Transform, Load (ETL) operations, enabling the extraction of data from various sources, its transformation, and its loading into different destinations [30].

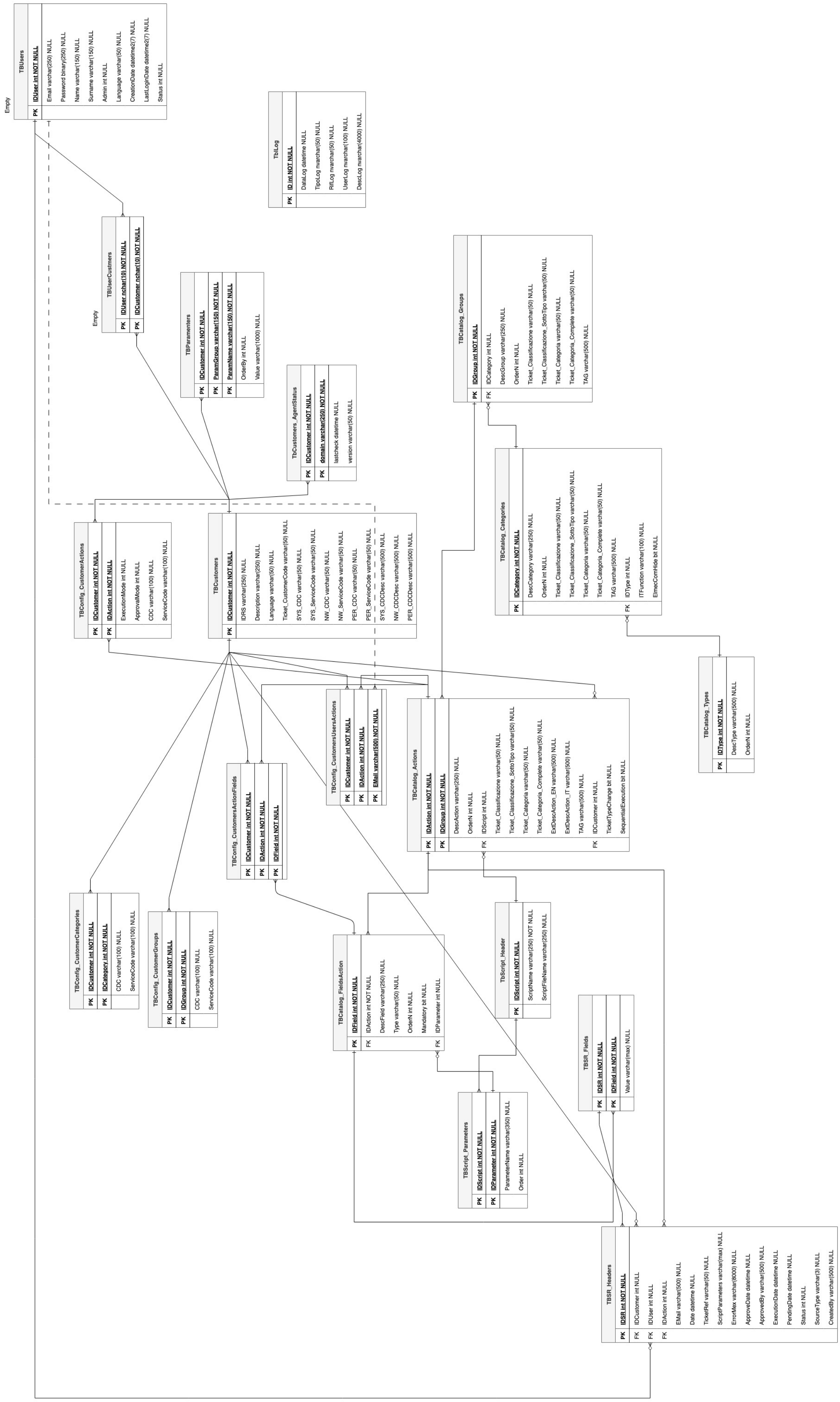

Figure 1.7: AS-IS Database ER Schema (TODO split into multiple smaller images)

database from guaranteeing data integrity [7]: for example, a row in a child table could refer to a non-existent row in the parent table [44]. Such a scenario, known as a "dangling reference[4]", can generate anomalies and inconsistencies in the system, making the data unreliable and debugging extremely complex [7].

Data Redundancy

Another issue is the high data redundancy and the large number of duplicate attributes across different tables [34]. This design, which deviates from the principles of normalization [2], leads to storing the same information in multiple locations, which not only increases the data volume but also introduces a high risk of inconsistencies [2].

Unclear and Inconsistent Naming Conventions

The naming convention for attributes and tables is inconsistent and unclear. The schema presents a mixture of conventions (e.g., `IDGroup` vs. `GroupID`, English names mixed with specific terms) and typos (e.g., `TBParamenters`), making it difficult to immediately understand the function of each field. In many cases, the naming is not explicit enough to describe the role of an attribute, requiring an in-depth knowledge of the system to correctly interpret the relationships and business logic. This lack of a unique and standardized vocabulary hinders collaboration among developers, complicates code maintenance, and increases the likelihood of errors.

1.3.4 Maintenance and Operational Challenges

According to information discussed in an internal meeting [27], the daily management of the system is made complex by several inefficiencies and obsolete design choices.

The current system design does not allow for the effective reuse of Service Requests and catalog configurations. Consequently, every modification or addition requires the creation of a new Service Request, even for small variations, leading to a innefficient and difficult-to-manage catalog, consisting of numerous duplicate SRs and fields. This, while functional, is not a scalable approach and leads to significant inefficiencies in catalog management itself. For example, if a Service Request needs to be modified for all clients who use

[4]The "dangling pointer" is a problem typically addressed in the context of low-level programming, where a pointer refers to a memory area that is no longer valid because it has been freed or because the pointer is used outside the context of the variable's existence to which it refers [16], but the same dynamic can also be applied in the context of databases, where the "pointers" are represented by foreign keys.

it, the operator will have to modify not only the SR in question but also all the duplicate SRs for each client, with the risk of forgetting a modification or making mistakes. This approach, although probably conceived to prevent error propagation, makes the catalog difficult to navigate and maintain.

The new architecture will need to address these challenges by introducing a more robust, versioned system based on modern software design principles and patterns.

Chapter 2

Architectural Redesign

Given the numerous critical issues that emerged from the analysis of the current system, it is clear that a thorough redesign is necessary. This redesign must be able to perform the system's current functions more efficiently and in an optimized manner, while also satisfying a series of requirements that did not exist when the system was developed several years ago, and which the current system cannot manage because it was not designed to do so.

2.1 New System Requirements

The system can be divided into three main components: **Catalog Structure and Management**; **Service Request Execution**, and **Master Data Import**. The requirements for each of these areas will therefore be analyzed.

2.1.1 Catalog Structure and Management

This is the most substantial component of the three, dealing with everything concerning the definition, maintenance, and configuration of the actions executable by each customer. This therefore also includes the management of form fields, customer-specific customizations, scripts, their parameters, the target machine address retrieval logic, and so on.

Before proceeding with the analysis of the requirements, a clarification on the terminology to be used is necessary: an '*Action*' is understood as the formal definition of an operation that a client can execute on its infrastructure; a '*Service Request*' (SR), on the other hand, is understood as the effective execution of an Action. If a parallel with OOP is to be drawn, the Action can be seen as a class, and the Service Request as an object, an instance of the aforementioned class.

Actions

The Action entity can be seen as the core of the catalog component, and it is composed of the following main elements:

- General metadata, such as the name, an extended description, tags, a type, a group, a category, and other information related to administrative and billing aspects;

- The fields of the corresponding form that must be filled by the client for its execution, including the relative logic tied to types, data validation, dependencies between various fields (e.g., fields that, once completed, "enable" other fields), autofill (e.g., fields whose values are autofilled with values from other fields), sensitive values, and so on;
- The script that will be executed in the customer's infrastructure to perform the effective required operation;
- Script parameters, which may be field values, but also additional parameters necessary for execution (e.g., AWS credentials, 365 credentials, etc...);
- The target machine on which the script must be executed.

Some of these dynamics are already implemented in the current system (with all the critical issues of the case, as already discussed in the previous chapter), while others are not. The crucial point, however, is that all these configuration aspects must be customizable for specific customers and for specific actions [27].

Given that the users of this system are very large clients (Fendi, Valentino, Lavazza, to name a few), it is easy to imagine that each of them has highly specific needs, which make direct reuse of Actions across multiple clients impossible. At the same time, however, the creation of infinite duplicated Actions that share almost everything, with only small differences (as happens in the current system [27]), must be avoided.

It is therefore necessary to implement a system that provides for the possibility of defining a sort of hierarchical structure, where a "parent" action is configured, which can be "inherited" by several child actions that will specify only the modifications, all without affecting the default action.

Fields

Together with actions, fields are central components on which a large part of the system is based. As already mentioned, fields are entities that can be extremely diverse, such as text fields, selects, multiple-choice selects, radio buttons, etc. In the current system, there are 86 different field types. This number is so high because, due to how the current system is built, the "type" information is the only way the backend communicates to the frontend what type of field is to be rendered [33]. In fact, there are many different types that share the same logic, perhaps simply using two different services to retrieve the data. The effective field typologies are therefore much fewer, and it is crucial that they are identified and generalized as much as possible.

The general idea is that the logic for fields must be moved to the backend, which must return all the necessary information for the frontend to render the form: from field types, to validation regexes, to error messages, to any external services to be contacted to retrieve the options list, or to validate the data, and so on [27].

All this must obviously be compatible with the customization requirements specified in the previous section.

Regarding customizations, a typical example scenario could be the following: a customer uses an action, but for some reason requires two additional fields, which will then be needed by the script that will in turn require two additional parameters. Furthermore, for some fields, the client in question uses different validation rules and specific autofill templates. These customizations must be possible without modifying the default action and fields.

Regarding "particular" field types, the system must provide for [27]:

- Select fields whose options are customer-specific (e.g., "Domain" field, which is a select with all the customer's domains);
- Select fields whose options are global for the entire system (e.g., "Country" field, which provides a list of states that are the same for all customers);
- Select fields whose options are customer-specific and to obtain them an external service must be contacted, with the possible option to search (e.g., "Server" field, which is a select with all the customer's servers, returned by the Atlantis CMDB service);
- Autofilled text fields, meaning their values are obtained through a templating system (Jinja) that uses the values of other fields (e.g., "Logon Name" field, populated by concatenating the user's first and last name separated by, for example, a dot);
- Text fields whose validation occurs through regex (e.g., "Date" field, which must comply with the standard format);
- Text fields whose validation occurs through an external service (e.g., "Logon Name" field, which requires contacting the relevant Active Directory service for validation that a user with the same logon name does not already exist);
- Fields with sensitive values, whose values will therefore need to be saved encrypted, using some symmetric encryption algorithm (e.g., "Password" field);
- "Mixed" fields, composed of multiple "sub-fields" with different characteristics (e.g., "UPN" field, which is composed of the first letter of the

first name, the last name, an at sign (`@`), and the value of a select whose options are obtained from an external API call);

It is important that the system is designed in a scalable way, so that if another field typology were to be added in the future, it can be implemented with the minimum possible effort.

Configuration and Execution Consistency

A system for monitoring the consistency of the catalog configurations [27] will be necessary, in order to prevent the creation of invalid configurations, which would then lead to errors during SR execution.

For example, if a `select` type field is specified for an action, whose options are obtained from customer-specific parameters, but those parameters have not been defined for that customer, the system must report this inconsistency through some alerting system (e.g., email, ticket, monitoring dashboard). Alternatively, if an action is configured to be executed on a certain type of machine but the field necessary to select it is not among the action's fields, the system must report this inconsistency.

Similarly, a system for monitoring the consistency of SR executions must be implemented, in order to be able to report any errors that may have occurred during execution, such as an SR stuck in an "intermediate" state (e.g., `NEW`) for too long, or an SR that started execution (i.e., in `PENDING`) but never reached completion (i.e., in `COMPLETED` or `ERROR`) within a certain time limit (e.g., 2 hours).

Additional script parameters

Another topic that must be managed is that of script parameters. In the old system, there was granular control over every single parameter, which was linked to an action's script and possibly to a field in the corresponding Service Request. In practice, all the action's fields with their respective values were passed as parameters to each script, plus some additional parameters not linked to any field (e.g., Exchange Server, AD Sync Server, various credentials). This dynamic of additional parameters occurred through the definition of parameters with the foreign key on the field set to `NULL`, which were then intercepted by the stored procedure relating to the generation of the parameter string which, with hard-coded `if` statements (e.g., `IF @ParamName = 'ServerExc' BEGIN ...`), were valued accordingly, retrieving the information with custom and non-standardized logic.

In the new system, it will be necessary to standardize these dynamics, finding a way to define the additional parameters that will be needed, beyond those of the fields, during action configuration, and to generalize the implementation logic as much as possible, so that if new "groups" of additional parameters (e.g., AWS credentials, O365 credentials, etc.) were to be added, it can be done in the most efficient way possible.

Target Machine Configuration

In the old system, Service Requests were executed exclusively on the customers' Domain Controllers. Specifically, each DC ran an agent, which periodically checked via *polling*[1] if there were any Service Requests in the `PENDING` state with a matching `domain` field.

This part of the architecture will also need to be redesigned, as it is inefficient and highly limiting by design [40]. Therefore, in addition to redesigning these dynamics, it will also be necessary to provide the option to choose, during the configuration of an Action, to execute the related SRs on other types of machines, such as internal Elmec workers, or other types of machines on the customer's network other than the Domain Controller (e.g., Exchange Server). It must also be possible to choose to execute SRs through the old flow, in order to continue supporting both systems, at least initially [27].

2.1.2 Service Request Execution

In the current system, the execution of SRs occurs according to this sequence of main events:

1. The frontend sends the request to create a new SR to the REST service (`SRPWebServiceREST`), with all the necessary data (completed fields, customer, action, etc.) [33];

2. The SR and the relevant field values are created in the database, setting the status to `NEW` [33];

3. An SSIS job, every minute, checks if there are SRs in the `NEW` state, and if any are found, they are processed, moving them to subsequent states (e.g., `WAITING FOR APPROVAL`, `PENDING`, `ASSIGNED`, etc.) depending on the action and customer configuration [28];

4. When the SR is in the `PENDING` state, it means it is ready to be executed by the agent on the customer's DC;

5. Every 10 seconds, the agent on the customer's DC checks if there are SRs in the `PENDING` state for its domain (through calls to the SOAP service (`SRPWebService`)) [34];

[1] *Polling* is a communication technique where a system or device periodically and actively samples the status of an external resource or device to check for readiness or new data.

6. If any are found, for each one, the corresponding script is downloaded, executed, and the SR status is updated to `COMPLETED` or `ERROR` depending on the execution outcome (through the SOAP service) [34];

It has already been discussed in the previous chapter how this flow has numerous criticalities, which make it inefficient and not very scalable. It will therefore be necessary to completely redesign it, taking into consideration the possibility of executing SRs on machines other than DCs, and the possibility of executing them also on machines internal to Elmec, and even virtual machines [27].

Elmec already has an internal *provisioning* system, called *Provision Prime*, which is responsible for executing Ansible playbooks on machines (internal, external, and also virtual) [23], and which could be leveraged for this purpose. It will therefore be necessary to analyze this system, and understand if and how it can be integrated into the new SR execution flow, in order to avoid the polling of agents on the DCs, and to have a generally more reliable and efficient system [27].

A constraint that must be respected, however, is that the action scripts are all in PowerShell, and it is not possible to modify them. This is because they are very numerous, have been developed over the years, have been tested and validated for every single client, and are maintained by different Elmec departments. Therefore, even if the SR execution flow is redesigned, the scripts must be executed as they are. This obviously does not apply to new scripts that will be developed in the future, which can be designed leveraging the new technologies [27].

2.1.3 Master Data Import

Another important component of the system is the one responsible for importing customer master data, necessary for populating some action fields (e.g., UPN domains, user accounts, groups, Organizational Units (OU)[2], databases, etc.). The main difficulty lies in the fact that this information physically resides on the customers' machines (large enterprise clients, with thousands of user accounts and groups, are being discussed), and therefore their retrieval in real-time is not immediate.

In the current system, this import occurs in a manner very similar to the SR scenario, but in reverse. It happens through an agent running on the customers' DCs, which periodically (hourly) (downloads and) executes a series

[2]In Windows Server, an Organizational Unit (OU) is a container within Active Directory (AD) used to logically group objects like user accounts, computer accounts, and security groups, similar to a folder in a file system [1].

of PowerShell scripts to retrieve the necessary information, and deposits them in a specific system folder (`D:\SRPortal\Anag\`) [34]. These files are then consumed by an SSIS job (one for each type of master data) that imports them into the database. The REST services then expose APIs to retrieve this information, which are used by the frontend to populate the action fields [33].

In the new system, it will be necessary to redesign this flow as well, in order to eliminate the dependence on agents on customer DCs. Even in this case, however, the PowerShell scripts cannot be modified [27], and must be executed as they are now. It will therefore be necessary to analyze how to integrate the execution of these scripts into the new flow, perhaps also leveraging Elmec's internal *provisioning* system (*Provision Prime* [23]) in this case, in order to obtain a more traceable, reliable, and efficient system [27].

2.2 New System Architecture

After gathering and defining the requirements for the new *ServiceRequestManager (SRM)* system through various meetings with stakeholders and analysis of the current system (SRP), it has been possible to design a new architecture for the system that allows for efficient and scalable satisfaction of these requirements.

The proposed new architecture for the SRM system, shown in Figure 2.1, is based on a **microservices architecture**, which allows for greater modularity, scalability, and ease of maintenance compared to the distributed-monolithic architecture of the current SRP system [10].

2.2.1 Main Components

The SRM system architecture is composed of the following main components.

Frontend

Two separate web frontends will be developed for end-users and system administrators. These frontends will communicate with the REST services via HTTP/HTTPS APIs.

Due to the business logic being moved to the REST services, the frontends will be lighter and easier to maintain, allowing for greater flexibility in implementing new functionalities and user interface improvements [21]. This also allows for the future integration of a mobile application, which can use the same APIs.

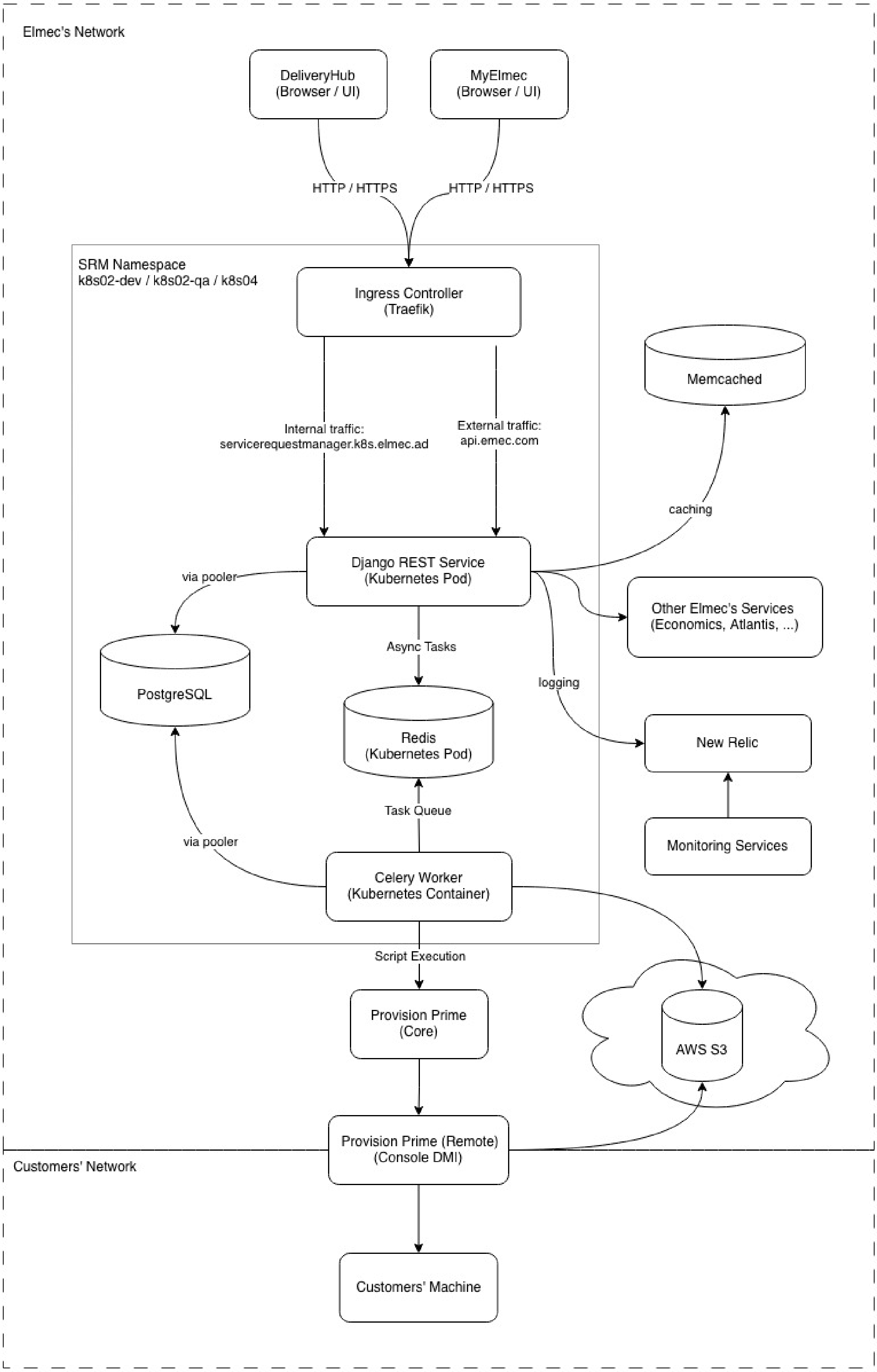

Figure 2.1: SRM System Architecture

REST Services

REST (Representational State Transfer) is an architectural style for designing web services that leverages HTTP protocols for communication between client and server [14]. The basic idea is that a system's resources (users, orders, documents, etc.) are exposed through a uniform interface, typically HTTP, using [14]:

- URLs to identify resources (e.g., `/api/catalog/actions/123/`);
- HTTP verbs to operate on resources (`GET`, `POST`, `PUT`, `DELETE`, ...);
- Representations of resources in standard formats (JSON, XML, ...).

These services will be responsible for business logic, user request management, and interaction with databases and other system components.

Authentication and Authorization System

Authentication and authorization will be managed through the internally developed library, PyElmecAuth or GoElmecAuth (depending on the implementation language), which will interface with the corporate identity management system to ensure secure and controlled access to the REST services.

It will therefore be necessary to integrate a granular permission system into SRM to ensure that users can only access the resources and functionalities for which they are authorized, based on the roles defined by the corporate Identity Provider.

Caching System

A caching system will be required to improve system performance and reduce the load on the databases.

In particular, responses from calls to external services will be cached, such as third-party APIs needed to validate some fields, or calls to retrieve customer information, to reduce response times and improve the user experience. The time-to-live (TTL) of the caches must be configurable based on the specific needs of each data type.

Message Broker and Task Queue

To manage asynchronous operations and improve system scalability, a Message Broker (e.g., RabbitMQ, Apache Kafka, Redis, ...) will be used in combination with a Task Queue (e.g., Celery).

This is necessary as the SR lifecycle is very complex and involves many operations that can require long execution times [26], such as sending emails, maintaining HDA[3] tickets, integration with external systems, etc. By using a messaging and task queue system, these operations can be executed in the background, without blocking the application's main flow [37].

Provisioning System

It will also be necessary to integrate a provisioning system to execute the scripts related to SRs and those for master data import directly within the clients' networks. This system must be secure, reliable, and scalable, in order to be able to manage a large number of provisioning requests simultaneously.

Elmec already has its own provisioning system called *Provision Prime* [23], which will be integrated with the new SRM system to execute scripts efficiently and securely.

Monitoring and Logging System

To ensure the stability and reliability of the system, a monitoring and logging system will be required. This system will allow tracking system performance, identifying any problems, and analyzing logs to resolve bugs. Tools such as Prometheus, Grafana, New Relic, or other similar tools will be used to implement this system.

Both system metrics (CPU, memory, database usage, API response times, etc.) and application logs (errors, warnings, debug information, etc.) will be tracked, in order to always have a complete view of the system status, and to have the possibility of launching automatic alarms in case of problems (e.g., if the number of responses resulting in 500 (Internal Server Error) in the last hour is $\geq 1\%$).

Containerization and Container Orchestration System

To facilitate system deployment, scalability, and management, a containerization system (Docker) will be used in combination with a container orchestration system (Kubernetes).

This will allow isolating the different system components, simplifying the deployment process, and ensuring greater resilience and scalability of the system, through functionalities such as replicas, chronjobs, automatic scaling of pods, and so on [19].

[3] HDA (Help Desk Advanced) is a web-based ticketing and service management system developed by Zucchetti. It enables organizations to manage tickets, incidents, and support processes through a centralized platform [17].

2.3 Addressing the Identified Challenges

All the challenges described in section 1.3 have been analyzed, addressed, and resolved through a series of targeted interventions. These interventions concerned various aspects of the system, from restructuring the software architecture and revising data management processes to implementing new technologies and development practices. The solutions adopted for each of the identified challenges are detailed below.

2.3.1 Architectural and Technological Challenges

The adoption of an approach based on containerization (with Docker) and orchestration (with Kubernetes, often abbreviated as K8s) forms the foundation for overcoming the current architectural and technological limitations of the SRP system. This transition not only directly addresses issues of modularity, isolation, and dependency management but also paves the way for a more modern and scalable architecture, such as microservices or micro-frontends. Specifically, it aims to solve the problems of fragmentation, dependency complexity, and maintenance difficulties inherited from the previous monolithic architecture and the agent installed at the customer's site.

Containerization with Docker

Docker is a containerization platform that allows developers to package applications and all their dependencies (libraries, configurations, environments) into a single, standardized unit called a container. As illustrated in figure 2.2, these containers are isolated from each other but share the host operating system's kernel, making them extremely lightweight and portable.

The use of Docker resolves the "works on my machine but not in production" problem (known as "dependency hell") [3] and provides a solid foundation for component decoupling. Each component (frontend, backend, support services) will be enclosed in a *Docker container*, and ensures that the execution environment (including libraries, configurations, and dependencies) is identical across development, testing, and production [3]. This eliminates errors caused by environmental discrepancies.

Once created, the container can be run anywhere Docker is present (in our case, container orchestration is managed by Kubernetes, which will be discussed in the next subsection), simplifying the deployment process. The system thus becomes extremely more portable, moving away from the strict dependence on specific operating systems and hardware configurations like, in this case, Windows Server and Microsoft SQL Server.

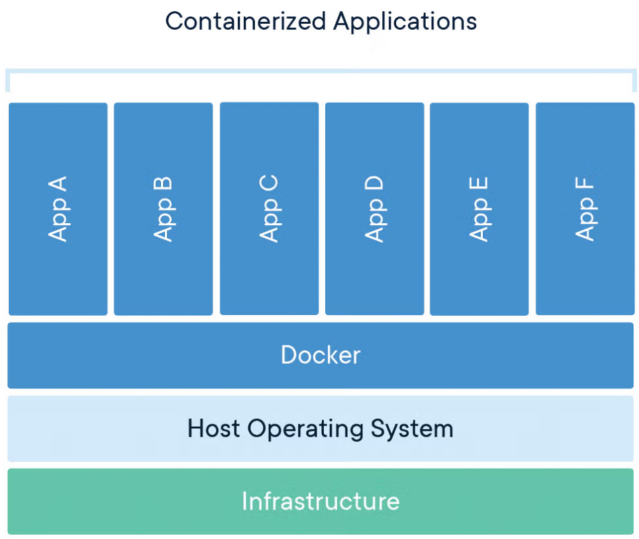

Figure 2.2: Docker Container Architecture (source: [43])

Orchestration with Kubernetes

Kubernetes (K8s) is an open-source system for automated container orchestration, originally developed by Google. Its purpose is to manage entire fleets of Docker containers in a *cluster*, ensuring that applications are always available, scalable, and resilient [19].

If Docker provides the mechanism for creating and distributing containers, Kubernetes is its natural complement, handling their large-scale management (*orchestration*), transforming the infrastructure from a set of single machines into a centrally managed cluster [19].

K8s allows components to scale horizontally based on the workload (via the Horizontal Pod Autoscaler). When demand increases, K8s automatically launches new instances of services (e.g., more pods for the REST backend) and removes them when the load decreases. This is crucial for managing traffic peaks [25].

In parallel, it constantly monitors the status of the containers. If a component fails, K8s automatically restarts it or schedules a new one on a healthy node in the cluster, ensuring high availability and resolving the single-point-of-failure problem introduced by the current dependence on single servers and the agent [25].

Kubernetes also supports advanced deployment strategies (e.g., Rolling Updates), allowing code updates without service interruptions (zero-downtime deployment) [25]. In case of issues, rolling back to the previous version is fast and automated. This mitigates the risk associated with system changes.

Finally, K8s also offers native mechanisms to manage environment variables and sensitive data (such as database credentials or external service authentication tokens) separately from the container code [25]. This centralizes management and increases security, eliminating the numerous hard-coded configurations present in the old system and making the containers more generic.

The adoption of Docker and Kubernetes therefore creates the ideal environment for the subsequent decomposition of the frontend monolith and the re-engineering of the various backend components, topics that will be covered in detail in the following sections.

Monolithinc Frontend Component

At a technological level, the chosen technology did not change compared to the existing system, which is Vue.js. However, Vue.js version 3 was used, which introduces numerous improvements compared to version 2 used in the existing system, including better performance management, a more flexible composition system, and improved support for TypeScript [42]. In fact, in addition to the updated version of Vue.js, it was decided to adopt TypeScript as the development language for the frontend, in order to improve code maintainability and quality [39].

TypeScript Developed in 2012 by Microsoft as free and open-source software (Apache License 2.0) [39], TypeScript extends JavaScript's functionalities by introducing optional static typing. The main advantage is that, unlike JavaScript, where type errors are detected only at runtime, TypeScript allows most errors to be caught already at the *build time* stage. This ensures greater code robustness and a significant reduction in production bugs. For extended projects like this one (the Service Requests component is actually only a part of a much larger customer-dedicated platform called *MyElmec*), static typing significantly improves code maintainability and readability, acting as a form of implicit documentation and allowing for safer and faster refactoring.

To solve the maintainability and scalability problems of the existing monolithic frontend, it was decided to move all the configuration and customization logic for field behavior (autofill, dynamic fields via external APIs, validation, dependencies, etc.) to the backend (a topic that will be explored in section 2.3.1). In this way, the frontend was transformed into a simple interface for display and user interaction (as it should be), significantly reducing code complexity and facilitating the implementation of new functionalities. In particular, the frontend is now limited to reading a JSON configuration provided by the backend, which describes the form structure, the fields to be displayed, their properties, and validation rules. This was also made possible thanks to the potential offered by Vue.js 3, which allows for the creation of highly dynamic and reusable components based on external data [42].

This led to a transition from a total of 86 different "types" of fields in the existing system to only 10 generic field types:

- Text
- Numeric
- File
- Password
- Checkbox
- Select (with already defined options)
- Select (with API-fetched options)
- Date / DateTime
- UPN (managed specifically due to its peculiarities, as it is composed of two sub-fields and has a particular validation logic)
- ADOU Tree

All the various customization logics (especially the customer-specific ones) thus become transparent to the frontend, which merely interprets the configuration provided by the backend. The system transitioned from a component with approximately 11,000 lines of code to a component with only 1,000 lines of code, resulting in improved maintainability and ease of system extension, and with the possibility of implementing other frontends (e.g., a mobile app) with minimal effort without having to replicate all the complex field management logic.

Overly Generic Backend REST Services

To solve the issues related to the frontend, a deep restructuring of the backend logic was necessary, especially concerning fields, moving from JSON responses of this type

```json
{
    "idAction": 455,
    "idField": 1649,
    "descField": "Manager",
    "type": "ADUser",
    "idCustomer": 42,
    "orderN": 53,
```

```
    "mandatory": false
}
```

to much more detailed and specific responses, which provide the frontend with all the necessary information for the complete rendering of the entire form, such as the following:

```
{
    "field": {
        "id": 3001,
        "name": "manager",
        "description": "Manager",
        "type": "select",
        "is_sensitive": false,
        "option_source_policy": "EXTERNAL_API",
        "external_api_config": {
            "name": "AD_USERS",
            "base_url": "https://q-api.elmec.com/srp/",
            "endpoint_pattern":
            ↪  "api/customer/VFG/adusers?domain={{domain}}",
            "http_method": "GET",
            "headers_json": {
                "Authorization": "{{TOKEN_TYPE}} {{TOKEN}}"
            },
            "response_mapping_json": {
                "name": "userDesc",
                "value": "user"
            },
            "timeout_seconds": 10,
            "is_searchable": true,
            "search_param_name": "search",
            "root_parent_id": null
        },
        "options": null,
        "multiple_choice": false,
        "file_accept_mimes": null
    },
    "mandatory": false,
    "validation_regex": "^[a-zA-Z0-9._]{5,20}$",
    "validation_regex_message": "Invalid format for Manager
    ↪  field.",
    "autofill_template": null,
    "depends_on": [
        "domain",
        "first_name",
        "last_name"
```

```
    ],
    "display_order": 26
}
```

In addition to modeling all possible properties of a field in a clear and structured way, the new REST services are also responsible for managing all the customization and configuration logic of the fields, with a three-level structure (which will be detailed in the chapter, dedicated to implementation aspects):

1. Fields
2. Actions
3. Customers

Broadly speaking, the idea is that each field (first level "Fields") has a set of generic properties (type, description, options, etc.) that can be further customized based on the specific action (second level "Actions") to which the field belongs (e.g., making it mandatory or not, adding validation rules, autofill logic, dependencies on other fields, etc.). Finally, each action can be further customized per customer (third level "Customers"), allowing the field's behavior to be adapted to the specific needs of each customer (e.g., modifying the available options in a selection field, changing validation rules, etc.), all without affecting the "original" configuration of the underlying level.

This three-level structure allows for a high degree of reusability and modularity, making it possible to define generic fields that can be easily adapted to different situations without having to duplicate code or logic.

Dependency on the SRP Agent

The dependency on the agent installed at the customer site has been completely eliminated thanks to the integration of Provision Prime, an internally developed provisioning tool written in Go, which manages the execution of Ansible playbooks on remote machines, in this case, the servers within customer networks, transparently to the developer (who will simply configure the playbook and make an API call).

Ansible Ansible is an open-source, agentless IT automation engine used for configuration management, application deployment, intra-service orchestration, and provisioning [4]. It is designed to be simple, powerful, and easy to use, operating over standard SSH for Linux/Unix and WinRM for Windows, thus requiring no special client software (agents) to be installed on the managed nodes [4]. This reliance on existing transport protocols and its human-readable structure, written in YAML, contribute to its minimal learning curve and rapid adoption. The core of Ansible's functionality are

Playbooks. A Playbook is essentially a blueprint of automation tasks, structured as a YAML file, that describes a set of steps to be executed on specified hosts or groups of hosts (defined in an inventory) [4]. They define the desired state of a system in a clear, declarative manner. Each Playbook consists of one or more plays, and each play is an ordered list of tasks. A task is a call to an Ansible module, which is the unit of code that performs a specific action, such as installing a package, copying a folder, or executing scripts. Playbooks ensure that complex, multi-tier application deployments and job executions are repeatable and reusable, transforming manual, error-prone processes into reliable, version-controlled automation.

In this way, not only is the dependency on the agent, and all the associated logic, eliminated, but all the scripts related to the Service Requests, which previously resided physically inside the customers' Domain Controllers and were not tracked in any way, are now versioned.

The cost of these improvements is represented by the additional logic needed to manage communications with Provision Prime, specifically the retrieval of the target machine. To implement these retrieval logics, which are highly custom for each type of target machine (e.g., Domain Controller, Exchange Server, internal workers, etc.), the *Strategy* design pattern [24] was leveraged. This pattern allows each retrieval logic to be encapsulated in a separate class, making the code more modular, extensible, and maintainable (a topic that will be explored in depth in chapter X.X).

As for the script execution logs, which are needed to monitor the status of Service Requests and detect execution errors, an integration with an AWS S3 bucket, a scalable and durable storage service, will be implemented. This way, every time a script is executed on a remote machine, the logs will be uploaded by the DMI (via the Ansible playbook) to the S3 bucket, ensuring centralized and secure access to execution logs regardless of the machine on which they were generated.

2.3.2 Code and Data Management Issues

The lack of versioning and modern debugging capabilities outlined in the existing system is fundamentally addressed by centralizing all disparate logic (previously scattered across Stored Procedures and SSIS jobs) into a unified, version-controlled software project integrated with a robust CI/CD pipeline. By adopting Git as the standard Version Control System (VCS), all business logic is now treated as application code. This immediately solves the versioning problem: every change is tracked, attributed to an author, and can be reverted, enabling seamless collaboration and a full audit trail. Moreover, moving the core business logic out of the database and into a dedicated backend service (Django REST Framework) written in a modern language (Python) drastically improves debuggability, allowing developers to use powerful, standardized IDE tools to set breakpoints, inspect variables, and step through code without impacting the production database.

This strategy is completed by integrating several specific CI/CD pipelines (a topic that will be explored in depth in chapter X.X) tailored for each environment branch (`feature`, `devel`, `quality`, `main`). This setup automates the deployment process: code changes are first pushed to a dedicated task branch, reviewed, and then merged into the `devel` environment. Upon passing, the code is merged to the `quality` environment, and finally to `main` for production release. This process ensures that no logic is directly modified in a production environment, eliminating the single-point-of-failure inherent in direct database and server access. Furthermore, replacing the inaccessible, unversioned, and tightly coupled SSIS Jobs with modern, containerized async tasks (managed as code within the Git repository) breaks the strong infrastructure coupling, enhancing accessibility, facilitating collaboration, and making maintenance and debugging a safe, environment-specific activity.

2.3.3 Database Schema and Data Integrity Issues

The issues related to database design and data management were addressed through a series of interventions aimed at improving data integrity, consistency, and maintainability. These interventions include a complete revision of the database schema to eliminate redundancies and inconsistencies, the implementation of stricter integrity constraints, and the adoption of clear and consistent naming conventions for tables and fields.

The system transitioned from using Microsoft SQL Server as the Database Management System (DBMS) to PostgreSQL, an open-source DBMS known for its robustness, scalability, and compliance with SQL standards. PostgreSQL offers advanced features such as support for complex data types, full ACID transactions, and an extensibility system that allows new functionalities to be added via extensions [35]. This choice not only improves the system's performance and reliability but also facilitates data management and the implementation of database design best practices.

Lack of Data Integrity and Consistency

To resolve data integrity and consistency issues, stricter referential integrity constraints were implemented using well-defined primary and foreign keys, also specifying behaviors in case of deletion (`ON DELETE CASCADE` / `SET_NULL`, etc...).

Specifically, for the deletion of "core" entities (such as customers, actions, or fields), the `CASCADE` behavior was chosen for "child" entities, which are consequently deleted.

For secondary entities and/or those that can be null (such as ticket categorization information, action categories and groups, order information associated with tickets, etc...), the `SET_NULL` behavior was chosen to preserve the history of operations and associated data, preventing the loss of more important information.

These constraints ensure that relationships between tables are correctly maintained, preventing the creation of orphaned or inconsistent data.

Data Redundancy

New relationships were created to maximize data reuse and reduce redundancy, while ensuring referential integrity through the use of well-defined primary and foreign keys.

These new entities primarily model concepts that were previously duplicated across multiple tables, such as cost centers (CDC, *"centro di costo"*), service codes, and ticket classification and category. This significantly reduced the amount of redundant data, improving storage efficiency and database maintainability.

Unclear and Inconsistent Naming Conventions

Now, all tables and their attributes follow standardized naming conventions (*snake_case*), improving the readability and maintainability of the database schema.

For "core" relationship names, the plural form was chosen (e.g., `actions`, `service_requests`, `customers`, etc...), while for "child" relationships, the singular name of the "parent" relationship followed by the specific name in the plural was chosen (e.g., `action_fields`, `customer_actions`, etc...).

2.3.4 Maintenance and Operational Challenges

With the new improvements introduced in this chapter concerning architecture, technology, the completely redesigned catalog and configuration management, and the new execution flow, many of the previously encountered maintenance and operational problems have been resolved. The standardization of development and deployment processes, the adoption of DevOps practices, and the implementation of a more modern and scalable infrastructure have led to a significant reduction in downtime, while improving the system's overall reliability. Furthermore, the adoption of advanced monitoring and logging tools allows for rapid identification and resolution of any issues, ensuring smooth and continuous system operation.

These improvements not only facilitate daily operations but also long-term planning, enabling the system to adapt easily to future needs and challenges.

Chapter 3

Implementation of a modular and scalable solution

While the previous chapter described in detail the requirements and the new architectural design for the new ServiceRequestManager (SRM) system, proposing targeted solutions to overcome the various technological and structural limitations of the legacy SRP system, this chapter marks the transition from the theoretical plan to practical realization, focusing on the concrete implementation of a modular and scalable solution.

The primary focus of this implementation lies in how the critical challenges described in the previous chapters were resolved. Specifically, it will be highlighted how the centralization of field management and configuration logic (and more generally of the catalog) on the backend eliminated the monolithic frontend and enabled a highly reusable three-level configuration system. The integration of the new execution and retrieval flow for master data through a message queue system will also be illustrated, which is essential for eliminating the dependency on the obsolete SRP Agent.

3.1 Automation of the Development Lifecycle through CI/CD Integration

For each feature or significant code modification, a dedicated branch is created named with the ID of the ticket associated with the modification (e.g., IN-7835). These branches are used for isolated development of new features, allowing teams to work in parallel without interference. Once the modification is complete, the branch is submitted to code review and automated testing through the CI/CD pipeline before being integrated into the main branch.

There are 3 distinct live environments: `devel`, `quality`, and `production`. The company Git workflow requires that feature branches be first merged into `devel` for further testing and integration, and subsequently into `quality` for final validation before production release. This approach ensures that each modification is accurately verified and tested in environments progressively closer to production, reducing the risk of introducing bugs or regressions into the live system.

3.1.1 Feat Branches

For feat branches, the CI/CD pipeline is designed to execute a series of automatic checks aimed at ensuring code quality, security, and compliance before its integration into the main branches, thus not including the build, test, and deploy phases of the application, as shown in figure 3.1.

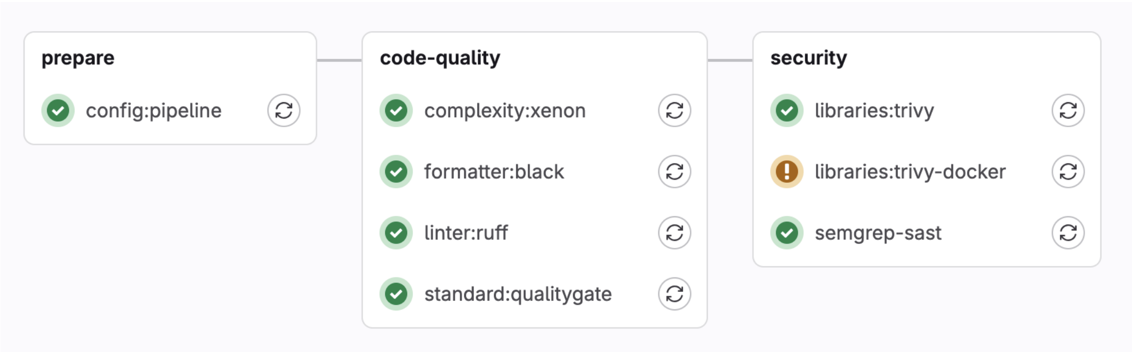

Figure 3.1: Feat Branches pipeline workflow

Prepare

The `config:pipeline` stage is responsible for preparing the execution environment by dynamically retrieving the project configuration from an external service. After installing the necessary tools, it executes an HTTP request to obtain the configuration parameters in JSON format and converts them into standardized environment variables, saving them in a .env file. This file is then published as an *artifact*, so that subsequent pipeline steps can use this information to adapt their behavior to the project-specific settings.

Code Quality

Complexity Check The `complexity:xenon` stage is responsible for measuring code complexity through the *Xenon* static analyzer, with the goal of identifying files or functions that might be difficult to maintain. After preparing the Python environment and installing the necessary tool, the job executes the analysis by comparing the results with a predefined quality threshold. The outcome is saved in a report file and sent to the project quality monitoring system. If critical issues that exceed the allowed level are detected, the job signals a failure, thus contributing to keeping software complexity within sustainable limits over time.

Code Formatter The `formatter:black` stage aims to verify that Python code complies with the formatting conventions established by the project (PEP-8), using the *Black* tool. During the job, the shared formatting configuration is downloaded, then a check is executed on the entire repository without applying automatic modifications. The analysis outcome is saved in a report and recorded in the quality monitoring system. If files that do not conform to the required style are found, the job signals an error to ensure that all modifications respect uniform code formatting standards.

Code Linter The `linter:ruff` stage is dedicated to static code checking in order to identify errors, unsafe constructs, or constructs not compliant with internal development guidelines. A Python linter (*Ruff*) is executed that analyzes the entire project and produces a report with any violations or refactoring warnings. The results are collected and sent to the quality monitoring system, contributing to maintaining a uniform style and preventing potential runtime problems. In case of errors considered blocking according to the project-defined rules, the job fails, preventing the integration of non-compliant code.

Quality Gate The `standard:qualitygate` stage statically analyzes the project through a set of rules that verify compliance with company standards. The system inspects Python files, YAML configurations, requirements, and deployment scripts, checking aspects such as proper debug configuration, presence of monitoring systems (*Sensu* / *Prometheus*), appropriate use of standard company libraries, APM (Application Performance Monitoring) configuration, and secure environment variable management. Each rule violation is reported with details about the file and the affected line, blocking the pipeline if critical errors are detected. This approach ensures that only code compliant with security and quality standards can be deployed to production, significantly reducing the risk of operational problems and vulnerabilities.

Security

Libraries The `libraries:trivy` stage is focused on dependency analysis in order to identify known vulnerabilities in packages used by the project. Trivy is used, a tool specialized in scanning libraries and software components against updated CVE databases [38]. The check result allows for timely identification of security risks related to versions affected by known bugs or flaws [38], thus contributing to maintaining an adequate security level and preventing the introduction of vulnerable dependencies in the release process.

The `libraries:trivy-docker` stage, instead, focuses on searching for misconfigurations in deployment files, Dockerfiles, and Kubernetes manifests. Trivy executes a complete scan using a "*comprehensive*" detection priority and generates a detailed report in JSON format, which is saved as an artifact to allow subsequent analysis. The scan also verifies dependency licenses and fails if critical unresolved vulnerabilities or significant misconfigurations are detected.

These two stages provide essential feedback on the project's security posture, allowing the team to proactively identify and correct potential security problems before deployment to production.

Static Application Security Testing (SAST) This stage executes static source code analysis to identify security vulnerabilities and potentially dangerous code patterns. *Semgrep* analyzes the code without executing it, detecting issues such as SQL injection, XSS, insecure credential management, and other OWASP vulnerabilities [29]. The stage generates a standardized report in JSON format that classifies vulnerabilities by severity (Critical, Medium, Info) and includes metadata such as commit SHA, branch, and project name. For main branches (quality, devel, master), the results are sent to a centralized project evaluation system for historical tracking. In this way, it is ensured that no code with known security problems can reach production environments.

3.1.2 Live Environment Branches

For live environments, namely those of devel, quality, and production, the CI/CD pipeline includes additional specific phases for building, testing, and deploying the application, as shown in figure 3.2, excluding the phases already covered for feat branches.

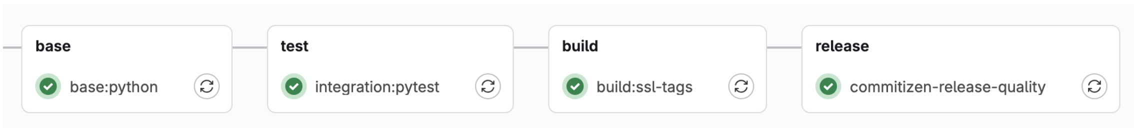

Figure 3.2: Live Environment Branches pipeline workflow (ecluding already covered stages)

Base Image Build

The `base:python` stage is responsible for building the project's base Docker image, containing all Python dependencies defined in the `requirements.txt` file.

This stage implements an optimized build strategy through layer separation: the image is rebuilt only when changes are detected in the `requirements.txt` or `Dockerfile_base` files, avoiding unnecessary rebuilds and significantly reducing pipeline execution times. The job is executed exclusively on main branches (devel, quality, master).

This separation into three distinct Dockerfiles (`Dockerfile`, `Dockerfile_base`, `Dockerfile_ssl`) allows for intelligent use of Docker cache: if dependencies don't change, subsequent stages can reuse the existing base image, significantly speeding up the build process and reducing the load on CI/CD runners.

Test

The `test:pytest` stage is responsible for executing the project's automated test suite using PyTest and PostgreSQL as the reference database. The job configures an isolated test environment by starting a dedicated instance of PostgreSQL 16 as a containerized service, with credentials and connection parameters dynamically defined through environment variables. Test execution occurs only if the `requirements_tests.txt` file is present and is adapted based on context: for devel and quality branches, the `devel-latest` and `quality-latest` Docker images are used respectively, while for merge requests the `production-latest` image is employed, thus ensuring that tests are executed in conditions as similar as possible to the target environment.

The stage also implements rigorous control over code coverage: if the coverage percentage is less than 80%, the pipeline automatically fails, ensuring that a minimum level of software quality and reliability is maintained through adequate test coverage. The stage is excluded from tags and the master branch to avoid redundant executions.

This approach ensures that every code modification is validated against a real database before integration, reducing the risk of regressions and compatibility problems.

SSL Image Build

The `build:ssl-tags` stage is dedicated to building the intermediate Docker image containing SSL certificates and security configurations necessary for the project's encrypted communications. This stage is part of the multi-layer build strategy and is executed exclusively on quality and master branches, only if the `Dockerfile_ssl` file is present. Similarly to the `base:python` stage, it implements a conditional rebuild system that reconstructs the image only when changes are detected in the `requirements.txt` or `Dockerfile_ssl` files, optimizing pipeline execution times through Docker cache reuse. This separation of the SSL layer allows for independent management of security configurations and certificates, facilitating updates and maintenance of encryption-related aspects without having to rebuild the entire application image.

Release

The `commitizen-release` stage manages the automatic versioning process and project release creation using *Commitizen*, a tool that analyzes commit history to automatically determine the version number according to Semantic Versioning[1] and Conventional Commit[2] conventions. The `commitizen-release` stage operates on the master branch and, after configuring Git credentials and synchronizing the local repository with the remote one, executes an automatic version bump based on conventional commit messages (feat, fix, core, refactor, test, ...). The process automatically creates corresponding Git tags and pushes them to the repository with the `ci.skip` option to avoid recursive pipeline activation. The `commitizen-release-quality` stage works similarly but is dedicated to the quality branch and generates pre-release versions with "alpha" suffix, dynamically modifying the Commitizen configuration to adapt it to the test context. Both stages are executed only if the `.cz.yaml` configuration file is present and are excluded from merge requests, ensuring that versioning occurs exclusively on main branches after code integration.

This automated approach eliminates the need to manually manage version numbers and ensures consistency and traceability in the software release process.

3.1.3 Tag Branches

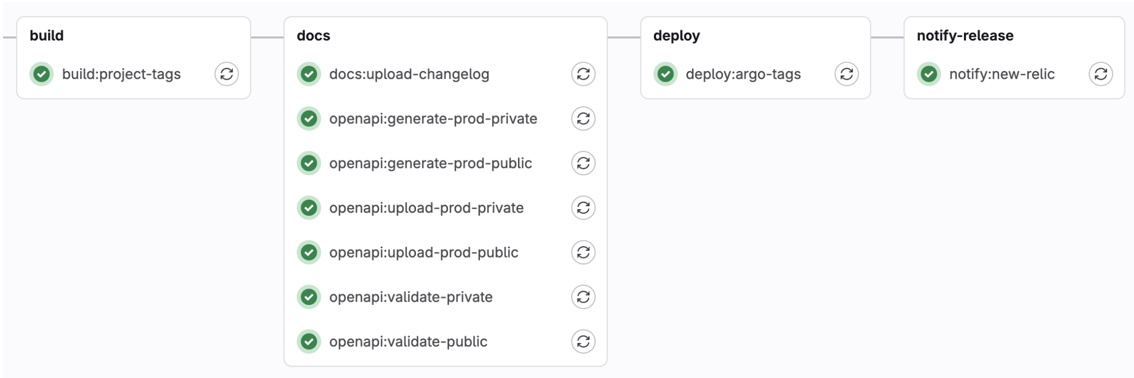

Figure 3.3: Tag Branches pipeline workflow (ecluding already covered stages)

Project Build

The `build:project-tags` stage represents the final phase of the build process, responsible for building the final Docker image of the application ready for deployment. This stage is activated exclusively when a Git tag respect-

[1]Semantic versioning is a versioning convention that adopts the MAJOR.MINOR.PATCH scheme, where the MAJOR number is incremented in case of changes incompatible with previous versions, the MINOR number following the introduction of compatible features, and the PATCH number for corrective interventions that do not alter existing functionalities.

[2]Conventional Commits is a convention for writing commit messages that uses a standardized structure composed of a modification type (feat, fix, chore, test, refactor, ...), an optional scope, and a description, in order to make change tracking, release note generation, and software versioning clearer and more automatable.

ing the Semantic Versioning format (major.minor.patch) is created, ensuring that only formally released versions are containerized. The job uses the main `Dockerfile`, which assembles all previously built layers (base and SSL), adding the application code and final configurations. The stage implements conditional logic that prevents execution if a separate `Dockerfile_base` exists, delegating in that case the build to more optimized flows that leverage multi-layer cache. The resulting image is tagged with the specific release version and published to the company Docker registry, constituting the definitive artifact ready to be deployed to production environments through subsequent pipeline stages.

Docs

The documentation stages automate the generation and publication of the project's technical documentation. The `docs:upload-changelog` stage is responsible for publishing the changelog on Techpedia, a company collaborative documentation platform, using the GraphQL API to automatically create or update changelog pages when a new release tag is created. The `openapi:generate` stages instead handle the generation of API documentation in OpenAPI/Swagger format, using the Django Spectacular framework to extract the specification from code annotations. These stages produce two versions of the documentation: a public one, containing only externally accessible endpoints, and a private one with complete documentation of all internal APIs. Generation occurs for each environment (production and quality) by connecting to the respective databases to ensure that the documentation accurately reflects the actual data structure. The generated OpenAPI schemas are saved as artifacts and can subsequently be used for automatic validation, SDK client generation, or publication on API documentation portals, ensuring that technical documentation is always synchronized with the code and accessible to developers and API consumers.

Deploy with ArgocD

The `deploy:argo-tags` and `deploy:argo-tags-quality` stages manage the automatic deployment of the application on Kubernetes environments through ArgoCD[3]. The `deploy:argo-tags` stage is activated exclusively for stable release tags (major.minor.patch format) and performs deployment to the production environment, while `deploy:argo-tags-quality` handles pre-release versions (with alpha, beta, rc suffixes) by deploying them to the quality environment for integration testing and validation. Both stages use a specialized Docker image containing candymand, an internally developed wrapper tool

[3]ArgoCD is a GitOps tool for Kubernetes that automatically keeps the state of applications in the cluster synchronized with that declared on Git.

that interacts with ArgoCD to automate the declarative deployment process. The tool receives as parameters the project namespace and the version to be deployed, then automatically updates the Git configurations of Kubernetes manifests, triggering the GitOps process managed by ArgoCD that synchronizes the desired state with the cluster.

This approach ensures fully traceable deployments, simplified rollbacks, and full adherence to Infrastructure as Code principles, eliminating manual interventions and reducing the risk of errors during release.

Notify Release

The `notify:new-relic` stage is responsible for automatically registering new application releases on the *New Relic* Application Performance Monitoring platform, creating a deployment marker that facilitates correlation between software versions and performance metrics. The job is activated for any tag that respects the Semantic Versioning format and uses the New Relic GraphQL API to identify the correct application through its GUID, automatically distinguishing between the production and quality instance based on the presence or absence of pre-release suffixes in the tag. Once the monitored entity is identified, the stage creates a change tracking event that records the deployed version, allowing operators and development teams to visualize in New Relic dashboards when deployments occurred and to correlate any performance anomalies or errors with specific software versions. This automatic notification mechanism eliminates the need for manual deployment registration and provides complete visibility on the application lifecycle, supporting troubleshooting activities, regression analysis, and evaluation of release impact on system stability.

3.2 Implementation of the Deployment Architecture on Kubernetes

The SRM deployment architecture on Kubernetes is designed to ensure scalability, resilience, and separation of responsibilities through an organization based on distinct deployments, each optimized for specific system functions.

As mentioned earlier, there are three separate live environments: `devel`, `quality`, and `production`. For each environment, a corresponding deploy file is defined. For simplicity, the production one (`production.yaml`) will be analyzed, knowing that the other files share exactly the same structure, except for the environment variables (which reflect their respective environments) and the resources, which for `devel` and `quality` are lower than their production counterparts, since those environments are used only internally and therefore need to handle exponentially lower workloads compared to the production environment.

The high-level structure of deployment files is as follows:

```yaml
namespace: servicerequestmanager
serviceAccount:
  create: TRUE
deployments:
  - name: static
    ...
  - name: backend
    ...
jobs:
  - name: sync-srp-catalog
    ...
  - name: sync-srp-sr-history
    ...
  - name: import-directory-data
    ...
```

The system is isolated within the dedicated `servicerequestmanager` namespace, ensuring logical separation of resources and facilitating permission and network policy management. The configuration provides for the creation of a dedicated service account, which allows pods to interact with Kubernetes APIs according to the *least privilege* principle, limiting permissions only to those strictly necessary for system operation.

3.2.1 Static Content Deployment

The `static` deployment manages the distribution of static content through a containerized Nginx web server, configured to serve frontend assets such as JavaScript bundles, CSS, images, and other static files. This deployment implements an optimized caching strategy that reduces the load on the backend and significantly improves response times for end users.

```yaml
  - name: static
    services:
      - name: static
        port: 80
    ingresses:
      - name: static
        public: False
        type: http
        host: servicerequestmanager.k8s.elmec.ad
        path: /static
        port: 80
    containers:
      - name: nginx
```

```yaml
        image: docker.elmec.com/.../ssl:production-latest
        port: 80
      - name: nginx-mtr-exp
        image:
         ↪ docker.elmec.com/.../nginx-prometheus-exporter:0.8.0
        port: 9113
        args:
          - "-nginx.scrape-uri"
          - "http://127.0.0.1:8080/nginx_status"
    replicaCount: 1
    monitoring:
      - name: nginx-metrics
        port: 9113
    imagePullSecrets:
      - name: docker-registry-secret
```

Container Configuration

The deployment consists of two containers running on the same pod.

The main `nginx` container uses the `ssl:production-latest` image, which includes SSL/TLS configurations and certificates necessary for secure communications. The server is configured to listen on port 80 within the cluster, while external access occurs through the ingress system that manages SSL termination. In parallel, the `nginx-mtr-exp` container runs the *Nginx Prometheus Exporter*, a specialized tool that exposes web server operational metrics in Prometheus-compatible format. This sidecar container connects to the `nginx_status` endpoint exposed by Nginx on port `8080` and translates native statistics into Prometheus metrics accessible on port `9113`, allowing the monitoring system to collect data on throughput, latency, errors, and connection usage.

Service and Ingress Configuration

The `static` Kubernetes service exposes the deployment on port `80`, providing a stable access point independent of the lifecycle of underlying pods. The associated ingress is configured as internal (not public) and routes requests coming from the host `servicerequestmanager.k8s.elmec.ad` with path `/static` to the service, implementing virtual hosting that allows segregating static traffic from API traffic.

Monitoring Integration

The deployment natively integrates monitoring through the definition of a *ServiceMonitor* that instructs Prometheus to scrape metrics exposed on port `9113`. This configuration allows automatic collection of operational telemetry without the need for additional manual configurations, ensuring continuous visibility on the state of the static content service.

3.2.2 Backend Application Deployment

The `backend` deployment constitutes the core of the architecture and manages the entire application logic through a multi-container organization that separates responsibilities between API server and asynchronous workers.

```yaml
- name: backend
  vault:
    name: 'secrets'
  services:
    - name: api
      port: 5000
      mirror: TRUE
    - name: api-ext
      port: 5000
      mirror: True
  ingresses:
    - name: api
      public: False
      type: http
      host: servicerequestmanager.k8s.elmec.ad
      path: /
      port: 5000
      middlewares:
        - name: cors
          spec:
            headers:
              accessControlAllowMethods:
                - "GET"
                - "OPTIONS"
                - "POST"
                - "PUT"
                - "DELETE"
                - "PATCH"
              accessControlAllowOriginList:
                - "*"
              accessControlMaxAge: 100
              accessControlAllowHeaders: ["*"]
    - name: api-ext
      public: True
      type: http
      host: api.elmec.com
      path: /servicerequestmanager/
```

```yaml
  port: 5000
  middlewares:
   - name: strip
    spec:
     replacePathRegex:
      regex: ^/servicerequestmanager/(.*)
      replacement: /$1
   - name: cors
    spec:
     headers:
      accessControlAllowMethods:
       - "GET"
       - "OPTIONS"
       - "POST"
       - "PUT"
       - "DELETE"
       - "PATCH"
      accessControlAllowOriginList:
       - "*"
      accessControlMaxAge: 100
      accessControlAllowHeaders: [ "*" ]
containers:
 - name: api
  ...
 - name: celery
  ...
replicaCount: 2
monitoring:
 labels:
  port: "5000"
  host: "localhost"
 subscriptions:
  - django
 type: PodAndService
 podmonitors:
  - name: django-hc
   port: api
   path: /ht/
   healthCheck: True
imagePullSecrets:
 - name: docker-registry-secret
```

Vault Integration for Secrets Management

The deployment is configured to integrate *HashiCorp Vault*[4] through automatic injection of secrets into the pod filesystem. The *Vault Agent injector* dynamically mounts secrets (database credentials, API keys, passwords) in the path `/vault/secrets/secrets`, making sensitive information available to containers without exposing them in environment variables or ConfigMaps. This approach implements the secrets management pattern according to Kubernetes security best practices, ensuring automatic credential rotation and complete audit trail of accesses.

API Container Configuration

The `api` container runs the Django application through the Gunicorn server, configured to handle HTTP requests on port 5000. The resource configuration implements a *Burstable Quality of Service*[5] model, with requests of 400m CPU and 1000Mi memory and limits of 800m CPU and 1200Mi memory respectively. This configuration allows the container to temporarily scale beyond requested resources during load peaks, while ensuring a minimum baseline guaranteed by the Kubernetes scheduler.

```yaml
containers:
  - name: api
    image: "docker.elmec.com/innovation/servicerequestmanager:
    ↪  PROJECT_VERSION"
    port: 5000
    cpuRequest: 400m
    cpuLimit: 800m
    memoryRequest: 1000Mi
    memoryLimit: 1200Mi
    envs:
      - name: ENVIRONMENT
        value: "PRODUCTION"
      - name: DJANGO_DB_HOST
```

[4]HashiCorp Vault is a system for securely managing secrets such as passwords, tokens, certificates, and API keys. It's used to centralize, encrypt, and dynamically deliver these secrets to pods, avoiding storing them in plaintext in YAML manifests or Kubernetes Secrets. It also enables secret rotation, temporary access with fine-grained control, and automatic secret injection into containers via sidecar or the CSI driver.[41]

[5]Burstable Quality of Service (QoS) is a resource classification applied to pods whose requested CPU/memory is lower than their limits. This means the pod is guaranteed at least its requested resources, but it's also allowed to "burst" up to its limits when the node has spare capacity. Burstable sits between Guaranteed (highest priority) and BestEffort (lowest priority) in terms of scheduling and eviction priority [22].

```yaml
        value: 'servicerequestmanager-db-production-pooler-rw.autom-servicerequestmanager-production.svc.cluster.local'
      - name: DJANGO_DB_PORT
        value: '5432'
      ...
```

Environment variables define the application's operational configuration, including PostgreSQL database connection parameters through the PgBouncer pooler, distributed caching configuration via Memcached, integration with Redis for Celery queue management, and external service endpoints for OIDC authentication and integrations with other company systems. The configuration also includes specific parameters for integration with the legacy SRP system, defining connections to the SQL Server database (`elmec-cmi-mssql.elmec.ad`) and synchronization dates for importing service request history. The S3-compatible storage system is configured through the *Ceph RGW* endpoint (`rgw.elmec.com`) for script-execution log management and customers' master data import/export.

Celery Worker Container

```yaml
- name: celery
  image: "docker.elmec.com/innovation/servicerequestmanager:PROJECT_VERSION"
  memoryLimit: 1200Mi
  memoryRequest: 1000Mi
  cpuRequest: 400m
  cpuLimit: 800m
  command: "/bin/sh"
  args:
    - -c
    - . /vault/secrets/secrets;cd servicerequestmanager; celery -A servicerequestmanager worker --loglevel=info --pool threads
  envs:
    - name: CELERY_QUEUE
      value: "servicerequestmanager_production"
    - name: REDIS_HOST
      value: "autom-servicerequestmanager-redis.autom-servicerequestmanager-production.svc.cluster.local"
    ...
```

The `celery` container runs Celery workers dedicated to asynchronous processing of background tasks related to the service request execution flow, using a thread pool to handle concurrent operations without blocking. The custom startup command initializes the environment by loading secrets from Vault before starting the worker with logging configured at INFO level. Workers are configured to consume from the dedicated `servicerequestmanager_production` queue on Redis, ensuring isolation from other systems sharing the same messaging infrastructure. Resource allocation is identical to that of the API container, reflecting similar computational requirements.

Service Exposure and Load Balancing

The deployment exposes two distinct services to handle internal and external traffic.

The `api` service operates internally to the cluster on port `5000`, configured with active mirror mode to allow traffic mirroring for testing or debugging purposes. The associated ingress is configured as internal and routes requests coming from the host `servicerequestmanager.k8s.elmec.ad` to the backend, implementing CORS middleware that permits cross-origin requests with complete HTTP methods (`GET`, `POST`, `PUT`, `DELETE`, `PATCH`) and arbitrary headers.

The `api-ext` service instead provides public access through the host `api.elmec.com` with path prefix `/servicerequestmanager/`. The ingress configuration includes a path rewriting middleware that removes the prefix before forwarding requests to the backend, allowing the application to operate with simplified routing paths. This ingress also implements permissive CORS policies to support integrations from third-party web applications.

Availability and Scaling

The deployment is configured with `replicaCount: 2`, ensuring redundancy and load distribution across at least two pods distributed on different cluster nodes. This configuration implements basic high availability, allowing rolling updates without downtime and tolerance to single pod or node failures. The Kubernetes scheduler automatically distributes replicas across different nodes when possible, maximizing resilience.

Health Checking and Monitoring

The monitoring system is configured through PodMonitor that combines application metrics and health checks. The `django-hc` PodMonitor executes periodic checks on the `/ht/` endpoint of the API service, verifying that the application is responsive and correctly configured.

The deployment is also tagged with django subscription that activates predefined dashboards and alerts for Django applications, providing immediate visibility on metrics such as request rate, latency distribution, error rate, and database connection pool utilization.

3.2.3 Scheduled Jobs

The architecture includes Kubernetes CronJobs for triggering Master Data export from clients' Domain Controllers and Exchange Servers, and for keeping data aligned between ServiceRequestManager and the legacy SRP system, so that the two systems can, at least initially, coexist and remain aligned during this period. The CronJobs share the same environment variables configuration and secrets injection as the backend deployment, ensuring consistency in access to databases and external services.

Here is an example of the CronJob definition configuration:

```yaml
jobs:
  - name: sync-srp-catalog
    vault:
      name: 'secrets'
    schedule: "0 1 * * *"
    image: docker.elmec.com/innovation/servicerequestmanager:PROJECT_VERSION
    memoryLimit: 700Mi
    memoryRequest: 400Mi
    cpuRequest: 400m
    cpuLimit: 800m
    command: "manage.sh"
    args:
      - "sync_srp_catalog"
    envs:
      ...
    imagePullSecrets:
      - name: docker-registry-secret
```

SRP Catalog Synchronization

The `sync-srp-catalog` job is scheduled for daily execution at 1:00 AM through the cron expression `0 1 * *` [6]. This job executes the Django management command `sync_srp_catalog` which queries the SRP database to import catalog configurations. Execution is configured with more contained

[6] The *cron* syntax is a compact format used to schedule periodic execution of processes or jobs in Unix-like systems [5]. It consists of five numeric fields separated by spaces (minute, hour, day of month, month, day of week) that indicate when a command should be executed. Each field can contain single values, ranges (e.g., 1-5), lists (e.g., 1,3,7), or special characters such as `*`, which means "any value", and `/`, which indicates a frequency (for example `*/5` for every five time units) [5]. A string like `0 2 * * *` therefore indicates the execution of a job every day at 02:00. This syntax, while minimal, allows for precise and flexible description of virtually any recurring scheduling need.

resources compared to permanent deployments (400Mi-700Mi memory, 400m-800m CPU), reflecting the batch nature of the workload that can tolerate higher latencies.

Service Request History Synchronization

The `sync-srp-sr-history` job is scheduled for execution at 2:00 AM, one hour after catalog synchronization to ensure that updated configurations are already available. This job executes the `sync_srp_service_requests` command which imports the history of service requests created in the legacy system starting from the date defined in `SRP_SR_SYNC_START_DATE`, keeping the unified view of requests aligned across the two systems during the transition period.

Master Data Import

The `export-master-ata` job is responsible for periodically triggering the export of customers' master data from their Domain Controllers and Exchange Servers via an Ansible Playbook. Scheduled to run every weeb day at 02:00 AM, this job executes the `import_master_data` management command.

This execution model based on Kubernetes Jobs ensures automatic retries in case of failure and retention of execution history for debugging and auditing.

3.3 Backend Service Implementation

Il servizio backend è sicuramente il componente più critico dell'architettura di SRM, responsabile della gestione della logica applicativa, dell'elaborazione delle richieste e dell'integrazione con sistemi esterni. L'implementazione è stata progettata per garantire scalabilità, resilienza e facilità di manutenzione, e un alto livello di personalizzazione per adattarsi alle esigenze specifiche dei clienti.

3.3.1 Technological Overview

Verrà fornita ora una panoramica delle principali tecnologie utilizzate nell'implementazione del backend di SRM.

Core Framework and API Layer

The backend service is built upon *Django*, a high-level Python web framework that follows the Model-View-Template (MVT) architectural pattern, a variant of the traditional Model-View-Controller (MVC) pattern adapted to web development. In Django's MVT architecture, the *Model* layer handles data structure and database interactions through an Object-Relational Mapping

(ORM) system, the *View* layer contains the business logic that processes requests and returns responses, and the *Template* layer manages presentation logic. Django acts as the implicit controller, routing URLs to appropriate views and coordinating the overall request-response cycle [11].

For RESTful API development, the framework is extended with *Django REST Framework* (DRF), a powerful toolkit that transforms Django into a complete API platform. DRF introduces additional architectural components such as serializers for data validation and transformation between complex types and native Python datatypes, viewsets that combine the logic for handling multiple related views, routers for automatic URL routing, and a comprehensive authentication and permission system [12]. The framework provides built-in support for content negotiation, pagination, filtering, and browsable API interfaces, significantly accelerating API development while maintaining consistency and best practices [12].

Django's *templates* are not used at all in this project, and therefore we can consider that the architecture effectively reduces to a classical MVC, where [6]:

- Model in MVC is the Django Model (Database/Business Logic).
- Controller in MVC is the Django View (APIView or ViewSet).
- View in MVC (Presentation) is handled by the Django Serializer / JSON Response (Data formatting and validation).

Technology Selection Rationale The choice of Django and Django REST Framework as the core backend technology was driven by several strategic considerations specific to the ServiceRequestManager project requirements. The primary alternative evaluated was *Go* with the *GORM* library, which would have offered superior raw performance and lower resource consumption due to Go's compiled nature and efficient concurrency model. However, Django was ultimately selected for the following reasons:

1. **Rapid Development and Time-to-Market**: Django's philosophy provides a comprehensive ecosystem of built-in components including ORM, authentication, admin interface, and form handling [11]. This significantly reduces development time compared to Go, where many of these functionalities would need to be implemented from scratch or integrated from third-party libraries. Given the project's requirement to replace the legacy SRP system within a defined timeline, development velocity was a critical factor.

2. **ORM Capabilities and Database Abstraction**: Django's ORM offers a sophisticated abstraction layer that handles complex database operations, query optimization, and migrations with minimal boilerplate code.

While GORM provides similar functionality [15], Django's ORM is more mature and includes advanced features such as automatic query optimization, `select_related` and `prefetch_related` for efficient eager loading, and a comprehensive migration framework.

3. **Admin Interface and Rapid Prototyping**: Django's automatically generated admin interface provided immediate value for system administrators to manage catalog configurations, user permissions, and service requests during development and early deployment phases. This eliminated the need to build custom administrative interfaces, which would have been necessary with a Go-based solution.

4. **Ecosystem Maturity and Library Integration**: The Python ecosystem offers extensive libraries for integration with external systems, including robust support for LDAP authentication, message queuing (Celery), data serialization, and enterprise system integration. Many of the required integrations with legacy systems and third-party services had well-maintained Python libraries, reducing integration risk and development effort.

5. **Asynchronous Task Processing Integration**: Django's native integration with Celery for asynchronous task processing aligned perfectly with the project's requirement for background execution of service requests and scheduled synchronization jobs. While Go's goroutines offer powerful concurrency primitives, implementing a comparable task queue system with persistence, retry logic, and monitoring would have required significant additional development.

While Go would have provided performance advantages, particularly for high-throughput scenarios, the anticipated load characteristics of the ServiceRequestManager system (primarily CRUD operations on catalog configurations and orchestration of external service request executions) did not justify the trade-off against development velocity and ecosystem advantages offered by Django.

Architectural Role Within the SRM architecture, Django serves as the central orchestration layer that coordinates all backend operations and system interactions. The framework's role encompasses several critical responsibilities:

- **API Gateway and Request Processing**: Through Django REST Framework, the backend exposes a comprehensive RESTful API that serves as the primary interface for the frontend application and external integrations [12]. DRF viewsets handle incoming HTTP requests, perform authentication and authorization checks, validate input data through serializers, and coordinate business logic execution. The framework's middleware stack implements cross-cutting concerns such as CORS handling, request logging, and exception management [12];

- **Business Logic Orchestration**: Django views and service classes implement the core business logic for catalog management, service request lifecycle handling, and configuration validation. The framework coordinates interactions between different system components, such as triggering asynchronous task execution through Celery, managing database transactions, and interfacing with external systems through integration adapters;

- **Data Persistence and Model Management** Django's ORM manages the complete data model [11], including catalog hierarchies, field configurations, service request instances, and audit trails. The framework handles database schema migrations [11], ensuring consistent evolution of the data model across `development`, `quality`, and `production` environments;

- **Authentication and Authorization** Django's authentication system, extended with OIDC integration, manages user identity verification and session handling [11]. The framework's permission system enforces role-based access control (RBAC) for API endpoints and data operations, ensuring that users can only access resources appropriate to their organizational roles;

- **Configuration Management and Multi-Tenancy** The three-level configuration system (Field, Action, Customer) is implemented through Django models and custom manager classes that handle inheritance and override logic. The framework manages the complexity of determining effective configurations by traversing the hierarchy and applying precedence rules, abstracting this complexity from both the frontend and external consumers;

- **Integration Hub** Django serves as the integration point for all external system interactions, including connections to the legacy SRP database for synchronization, LDAP servers for directory integration, S3-compatible storage for master data import / export, and message queue systems for asynchronous processing. The framework's modular architecture allows these integrations to be implemented as reusable components that can be tested and maintained independently.

This central architectural role makes Django the foundation upon which the entire ServiceRequestManager backend is built, coordinating the complex interactions required to deliver a scalable, maintainable, and feature-rich service request management platform.

Asynchronous Task Processing

The ServiceRequestManager architecture implements asynchronous task processing through *Celery*, a distributed task queue system designed for handling time-consuming operations outside the request-response cycle of web applications [8].

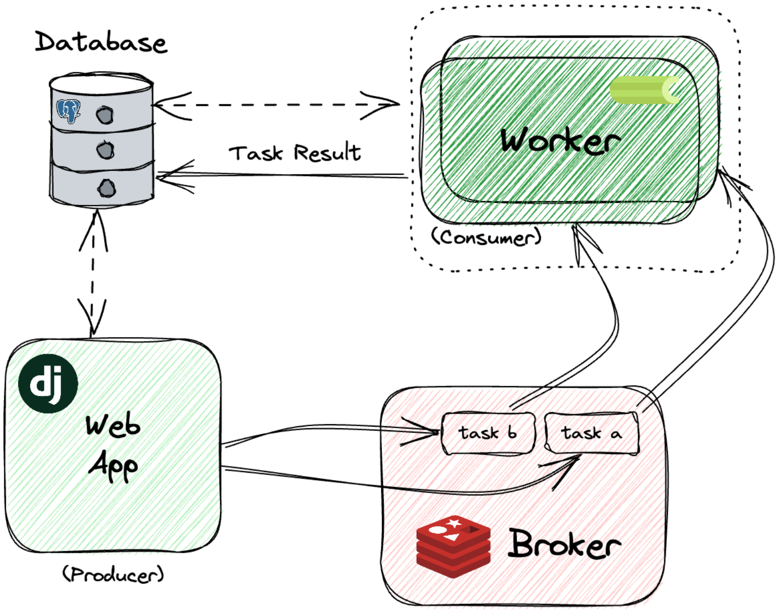

Figure 3.4: Celery Architecture (source: [20])

Celery, as shown in Figure 3.4, operates on a *producer-consumer* model where the Django application acts as a producer, publishing tasks to a message broker, while dedicated worker processes consume and execute these tasks asynchronously. This architectural pattern prevents blocking operations from impacting API response times and enables horizontal scalability of background processing capacity independently from the web server layer [8][20].

Celery Architecture and Components Celery's architecture consists of several key components that work together to provide reliable asynchronous execution. The *task* represents a unit of work defined as a Python function decorated with Celery's task decorator (`@shared_task`), which can be invoked asynchronously from anywhere in the Django application. When a task is called using the `.delay()` or `.apply_async()` methods, Celery serializes the function name and arguments, then publishes this message to the broker [8].

The *broker* acts as a message transport layer, receiving task messages from producers and delivering them to available workers. For the SRM system, *Redis* was selected as the message broker due to its *in-memory* architecture that provides low-latency message delivery, simple deployment and operational management, and native support for persistence through snapshots and append-only file (AOF) mechanisms. While *RabbitMQ* represents a robust alternative with more sophisticated routing capabilities, guaranteed message delivery through acknowledgments, and better performance under high-throughput scenarios with complex routing requirements, Redis was deemed more appropriate for SRM's use case. The system's task patterns are relatively straightforward, consisting primarily of service request execution orchestration and periodic synchronization jobs, without requiring advanced routing or particular patterns.

The *worker* processes continuously monitor the broker for new tasks, deserialize incoming messages, and execute the corresponding functions. As shown in the deployment configuration, Celery workers in the SRM system are configured with a thread pool (`-pool threads`) to efficiently handle I/O-bound operations such as API calls to external systems, database queries, and remote script execution coordination. Workers can be scaled horizontally by increasing the number of pods in the Kubernetes deployment, providing elastic processing capacity that adapts to workload demands.

Celery also includes a *result backend* for storing task execution outcomes, enabling the application to query task status and retrieve return values. This proves essential for long-running service request executions where users need to monitor progress and retrieve final results asynchronously.

Role in SRM Architecture Within the ServiceRequestManager system, Celery fulfills several critical functions that enable the asynchronous execution model required by the service request workflow:

- **Service Request Execution Orchestration**: When a service request is submitted through the API, Django immediately returns a response acknowledging receipt while delegating the actual execution to a Celery task. This task coordinates the multi-phase execution process described in Figure X.X, rendering execution scripts with related parameters, invoking Ansible playbooks, monitoring execution progress, collecting logs, and updating the service request status (and related ticket) in the database. This asynchronous pattern ensures that API endpoints remain responsive even when service requests take minutes or hours to complete.

- **Scheduled Synchronization Operations**: The periodic jobs defined in the Kubernetes CronJobs (`sync-srp-catalog`, `sync-srp-sr-history`, `export-master-data`) are implemented as Celery tasks triggered by scheduled task execution. This approach provides consistent error handling, retry logic, and logging across both user-initiated and scheduled operations, leveraging Celery's built-in support for task retries with exponential backoff and comprehensive task lifecycle monitoring.

- **Background Data Processing**: Operations such as bulk catalog imports, report generation, and data export to external systems are executed asynchronously through Celery tasks to avoid blocking interactive user operations. This maintains a responsive user experience while enabling resource-intensive batch processing.

- **Distributed Processing and Fault Tolerance**: Celery's distributed architecture ensures that task execution continues even if individual worker pods fail or are restarted during Kubernetes rolling updates. Tasks in the queue are automatically redistributed to healthy workers, and failed tasks can be configured to retry automatically, providing resilience against transient failures in dependent systems.

The integration of Celery with Django and Redis creates a robust asynchronous processing pipeline that is essential for handling the complex, medium/long-running operations inherent in automated service request management while maintaining the responsiveness and reliability expected of enterprise systems.

Data Persistence and Caching

The SRM architecture implements a two-tier data access strategy combining *PostgreSQL* for persistent storage and *Memcached* for distributed caching, optimizing both data durability and system performance.

PostgreSQL as Primary Data Store PostgreSQL serves as the system's primary relational database, managing all persistent data including the complete catalog hierarchy (actions groups, categories, tyhpes, and global, client, and action configurations), field definitions and validation rules, customer master data registries (Domain Controllers and Exchange Servers), service request execution history with associated metadata and logs, user accounts and RBAC permissions, and synchronization state for legacy SRP system integration.

The choice of PostgreSQL over alternative relational databases was driven by its advanced features particularly relevant to SRM requirements. PostgreSQL's robust transaction management with ACID guarantees ensures data consistency across complex operations involving multiple related entities, such as service request creation with cascading catalog validations and audit trail generation. The database's sophisticated query planner and support for partial indexes, expression indexes, and Common Table Expressions (CTEs) optimize performance for hierarchical catalog queries and configuration inheritance resolution.

As shown in the deployment configuration in Figure 2.1 and in section 3.2.2, the application connects to PostgreSQL through *PgBouncer* (`servicerequestmanager-db-production-pooler-rw`), a *connection pooler* that maintains a pool of persistent database connections, reducing connection overhead and enabling the application to handle connection spikes during peak loads without exhausting database connection limits. This architecture supports horizontal scaling of application pods while maintaining efficient database resource utilization.

Memcached for External Service Caching Memcached implements a distributed in-memory caching layer specifically targeting responses from external financial and administrative systems. These external services provide economic information required for service request processing, including active customer contracts with associated service levels and pricing, cost centers for departmental charge-back allocation, project codes for time and resource tracking, and general ledger account mappings for financial integration.

External service calls typically exhibit high latency due to network round-trips and complex business logic execution in source systems, while the data they return has temporal stability (contracts and organizational structures change infrequently). Memcached addresses this performance bottleneck by caching service responses with configurable TTL (Time To Live) values (usually one or two days), reducing redundant external calls and significantly improving API response times for catalog configuration retrieval and service request execution workflows.

Memcached's distributed architecture allows the cache to scale horizontally across multiple nodes, with consistent hashing ensuring efficient key distribution and lookup. The cache-aside pattern is implemented in Django service classes, which first query Memcached for cached data, invoke external services only on cache misses, and populate the cache with retrieved data for subsequent requests. This approach provides transparent caching without requiring modifications to external systems or introducing cache invalidation complexity for data managed outside SRM's control.

The combination of PostgreSQL for durable, transactional storage and Memcached for ephemeral, high-performance caching creates a data access architecture that balances consistency requirements with performance optimization, supporting both the complex relational queries required by catalog management and the low-latency access patterns demanded by interactive user workflows.

Remote Execution

The actual execution of service requests on customer infrastructure is orchestrated through *Ansible*, an agentless automation platform that provides idempotent, declarative configuration management and remote execution capabilities. Ansible was selected as the remote execution engine for its ability to manage heterogeneous Windows and Linux environments through a unified automation framework, eliminating the dependency on the legacy SRP Agent that previously required dedicated software installation and maintenance on customer systems.

Ansible Architecture and Integration Ansible operates on a push-based model where the control node establishes SSH or WinRM connections to target hosts (customer Domain Controllers and Exchange Servers) and executes tasks defined in YAML-based *playbooks*. The platform's agentless architecture requires only standard remote management protocols on target systems, significantly reducing deployment complexity and security surface compared to agent-based solutions.

For the SRM system, custom Ansible playbooks have been developed to encapsulate two primary categories of operations. The first category comprises service request execution playbooks that implement the automated provisioning workflows for user account creation, mailbox management, group membership modifications, and other directory service operations. Currently, these playbooks serve as wrappers that invoke existing PowerShell scripts developed by the Managed Systems team, providing a transitional architecture that maintains compatibility with established automation assets while introducing the Ansible orchestration layer. The playbooks are templated with Jinja2, receiving customer-specific parameters (domain names, organizational units, credential references) from the Django backend, then executed remotely through Ansible's `win_shell` or `win_powershell` modules.

The second category encompasses master data export playbooks responsible for extracting directory information from customer Active Directory and Exchange environments. These playbooks execute periodic exports of user accounts, security groups, organizational units (OUs), password policies, and UPN suffixes from Active Directory, as well as mailbox databases and distribution groups from Exchange Server. The exported data is serialized to JSON format and stored in S3-compatible object storage (MINIO, covered in the next section), creating the master data repository that feeds service request parameter validation and auto-completion features in the frontend application.

Execution Flow and Error Handling When a service request is submitted, the Django API creates a database record and enqueues a Celery task that orchestrates the Ansible execution. The worker process retrieves customer-specific connection parameters (target hosts, credentials from Vault, domain configurations) from the database, renders the appropriate playbook with request parameters, and invokes Ansible through a dedicated company internal service (*Provision Prime*). Execution output, including stdout, stderr, and structured result data, is captured in real-time and stored both in the PostgreSQL database for status tracking and in S3 for comprehensive audit trails and troubleshooting.

Ansible's idempotent task design ensures that playbook execution can be safely retried in case of transient failures (network interruptions, temporary service unavailability) without causing duplicate operations or inconsistent state. The platform's built-in error handling and conditional execution logic enable sophisticated workflows that adapt to environment-specific conditions, such as detecting whether a user account already exists before attempting creation or validating group membership before modifications.

Migration Strategy The current hybrid architecture, where Ansible invokes legacy PowerShell scripts, represents a deliberate migration strategy that balances risk management with modernization objectives. This approach allows immediate elimination of the SRP Agent dependency while preserving proven automation logic accumulated over years of production operation. The long-term roadmap envisions complete migration to native Ansible modules (such as `win_domain_user`, `win_domain_group`, and community Exchange modules), which will provide superior error handling, better idempotency guarantees, and more granular control over directory service operations. The modular playbook structure facilitates this gradual transition, enabling incremental replacement of PowerShell-based tasks with Ansible-native implementations without disrupting production service request processing.

This Ansible-based remote execution architecture provides a flexible, maintainable foundation for automated service delivery while establishing a clear path toward full infrastructure-as-code practices for customer environment management.

Object Storage

The SRM architecture leverages S3-compatible object storage for managing master data exports and execution artifacts, utilizing *Ceph RGW* (RADOS Gateway) as the underlying storage infrastructure exposed through the MinIO S3-compatible API. This object storage layer serves as an intermediary persistence mechanism between the remote export operations on customer infrastructure and the relational database, implementing a pattern that optimizes both network efficiency and query performance.

Storage Architecture and S3 Compatibility Ceph RGW provides an S3-compatible RESTful interface that implements the standard Amazon S3 API, enabling the application to use standard S3 client libraries (specifically the `boto3` Python SDK) without vendor lock-in to a specific cloud provider (AWS). The storage system is accessed through a dedicated endpoint, with bucket organization segregating data by type and customer. Master data exports are stored in structured JSON format with hierarchical key naming schemes (e.g., `customers/{client_id}/active_directory/users.json`, `customers/{client_id}/exchange/mailbox_databases.json`) that facilitate efficient retrieval and versioning.

Object storage was selected for master data management over direct database insertion for several architectural reasons. First, it decouples the export operation from database availability, allowing Ansible playbooks to complete successfully even during database maintenance windows or connection issues. Second, it provides a natural archival mechanism where historical exports can be retained with configurable lifecycle policies, supporting audit requirements and temporal queries. Third, it enables atomic updates of complete datasets, where the entire user registry or group listing can be replaced transactionally by uploading a new object, avoiding complex differential synchronization logic.

Master Data Import Pipeline The master data lifecycle follows a three-stage pipeline: export, storage, and import. During the export phase, Ansible playbooks execute queries against customer Active Directory and Exchange environments, serialize results to JSON, and upload the generated files to designated S3 buckets. These exports are scheduled to run periodically (as shown in the `export-master-data` CronJob executing weekdays at 02:00 AM), ensuring that the master data repository reflects recent organizational changes.

The import phase is triggered either on-demand or through scheduled jobs that enumerate available exports in S3, download the JSON files, and populate corresponding database tables. This import operation parses the structured data and creates or updates records for users, groups, organizational units, and other directory entities in PostgreSQL. By materializing this data in the relational database, the system achieves two critical performance optimizations: frontend components can query master data through efficient database queries with filtering, pagination, and search capabilities, significantly reducing latency compared to on-demand LDAP queries or API calls to customer infrastructure; and backend validation logic can rapidly verify service request parameters (such as confirming that a specified user exists in the target domain or that a requested distribution group is available) without incurring the network and authentication overhead of real-time directory service queries.

Execution Artifact Storage Beyond master data, S3 storage also serves as the repository for service request execution logs and artifacts. When Ansible playbooks execute, detailed output including task results, timing information, and any error messages are captured and uploaded to S3 with keys associating them to specific service request instances. This approach provides durable, cost-effective storage for audit trails without inflating the relational database with large text payloads, while still maintaining the ability to retrieve and display execution logs on-demand through presigned URLs or direct object retrieval.

The integration of S3-compatible object storage creates a scalable, resilient data pipeline that bridges the gap between distributed customer infrastructure and centralized SRM data management, enabling efficient master data synchronization while maintaining performance characteristics suitable for interactive user workflows.

3.3.2 General Project Structure

The ServiceRequestManager backend is implemented as a Python 3.13 project utilizing Django 5.2 and Django REST Framework 3.16, organized following Django's application-based architecture that promotes separation of concerns and modularity [11].

Django Application Organization

The project is decomposed into six specialized Django applications, each encapsulating a distinct domain of system functionality:

- `src/catalog/` manages all catalog-related logic, including Actions, Fields, customer configurations, validation regex patterns, API configurations, execution parameters, and script templates. This application implements the configuration hierarchy and provides the APIs for catalog CRUD operations.

- `src/integration/` handles integrations with external company systems such as HDA (ticketing), Economics (financial data), Atlantis (CMDB), and MySupport, as well as specialized integration endpoints for systems consuming SRM services outside the standard MyElmec/DeliveryHub workflow.

- `src/execution/` implements the complete service request execution lifecycle, including field validation, ticket creation, service request state management (creation, approval, suspension, execution, completion), and orchestration of Ansible playbook invocations.

- `src/monitoring/` provides health check implementations using Django Health Checks framework to verify catalog configuration consistency, service request state integrity, and synchronization status with external systems like HDA tickets.

- `src/directory_data/` manages the master data import pipeline, handling the ingestion of directory information from S3 storage into the relational database and providing query interfaces for user, group, and organizational unit lookups.

- `src/common/` contains shared utilities, generic controllers, and reusable components leveraged across multiple applications, reducing code duplication and promoting consistency.

Build and Deployment Artifacts

The `build/` directory contains the multi-layer Docker build configuration previously described, including `Dockerfile`, `Dockerfile_ssl`, and `Dockerfile_base`, along with production runtime scripts. The `run.sh` script orchestrates application startup by loading secrets from Vault, configuring SSL certificate trust, and launching the Gunicorn WSGI server with Eventlet worker class for async I/O support. The server is configured with 10 workers, 40-second timeout, and integration with New Relic APM for performance monitoring. The `manage.sh` script provides a wrapper for Django management commands, ensuring proper environment configuration and APM instrumentation during administrative operations and scheduled jobs.

The `deploy/` directory contains the Kubernetes manifests (`devel.yaml`, `quality.yaml`, `production.yaml`) defining the complete deployment configuration for each environment, including deployments, services, ingresses, CronJobs, and resource specifications as detailed in the previous deployment architecture section.

This organizational structure provides clear boundaries between functional domains, facilitates parallel development by different team members, and enables independent testing and evolution of each component while maintaining cohesive integration through well-defined interfaces.

3.3.3 Catalog Management

The catalog management subsystem represents the architectural core of ServiceRequestManager, implementing a sophisticated three-level configuration hierarchy that enables high flexibility in defining and customizing service request configurations. The catalog is composed of three principal entities that form the foundation of the system's automation capabilities: *Customers*, representing organizations enabled to use SRM; *Actions*, representing formal definitions of executable operations with complete configuration metadata; and *Fields*, representing configurable input parameters that users populate when creating service requests.

Three-Level Configuration Architecture

The catalog implements a hierarchical override system organized across three distinct levels, each providing progressively more specific customization capabilities:

Level 1: Global Definitions The base level consists of global Action and Field definitions (`Action`, `Field`, `ActionField` models) that establish default configurations applicable across all customers. An Action at this level defines fundamental properties including execution scripts, approval requirements, target infrastructure (Domain Controller, Exchange Server, internal worker, ...), and associated fields. Fields define input validation rules, data types (text, select, file, checkbox, etc.), regex patterns, and dependency relationships with other fields. The `ActionField` association model links fields to actions while allowing action-specific overrides of field properties such as validation rules or display characteristics.

Level 2: Customer-Level Action Customization The `CustomerAction` model enables customer-specific modifications to action properties without duplicating the entire action definition. Through this mechanism, individual customers can override execution scripts to accommodate environment-specific requirements, modify approval workflows, alter execution target types, or adjust additional script parameter groups. This level maintains a clear separation between shared action logic and customer-specific variations, eliminating the configuration duplication that plagued the legacy SRP system.

Level 3: Customer-Action-Specific Field Customization The `CustomerActionField` model provides the finest granularity of customization, allowing modifications to individual field behavior for specific customer-action combinations. This enables four critical operations: modifying existing field properties (validation regex, mandatory status, dependencies) for a specific customer and action; adding fields not present in the default action definition; removing fields from an action by setting `hidden=True`; and overriding field characteristics already customized at the action level through `ActionField`.

The configuration resolution follows a strict precedence hierarchy where `CustomerActionField` overrides `ActionField`, and `CustomerAction` overrides `Action`. This precedence is transparently applied by the retrieval methods, ensuring that the frontend receives the effective configuration without requiring awareness of the underlying hierarchy.

Service Block-Based Action Association

Action availability for customers is determined through a service block mechanism that replaces the granular one-to-one associations used in SRP. Each customer possesses a set of service blocks derived from contract information retrieved from the Economics system. Actions are associated with one or more service blocks, and a customer has access to all actions whose service blocks intersect with those in their possession. This set-based association model significantly reduces administrative overhead while providing flexible commercial packaging of service capabilities.

The system also supports direct customer-action associations for edge cases where a customer requires a highly specialized action not appropriate for service block classification. These customer-specific actions are visible only to the designated customer and operate outside the service block framework. For backward compatibility during the transition from SRP, the system maintains support for legacy granular associations through the `use_service_blocks` query parameter on relevant API endpoints.

Field Type System and Validation

The catalog implements a comprehensive field type system with specialized validation and behavior for each type. Simple types include text, textarea, password, email, numeric, integer, telephone, datetime, and date fields, each with default validation regex patterns that can be overridden. Complex types include text lists (arrays of text values with configurable minimum and maximum cardinality), selects (single or multi-select dropdowns), UPN fields (composite username@domain inputs), organization unit trees (hierarchical AD OU selection), and file uploads (with MIME type restrictions and file count limits).

Select fields support three distinct option source policies: options from customer-specific parameters stored in the Economics system, accessed by parameter group and optional name filter; options from global parameters independent of customer context; and options from external API calls configured through `ExternalAPIConfiguration` objects that specify endpoints, request parameters, and response mapping logic. The API configuration system supports Jinja2 templating in endpoint patterns, enabling dynamic URL construction based on customer IDRS or other field values.

Templating System for Dynamic Configuration

The catalog leverages Jinja2 templating throughout the configuration layer to enable dynamic behavior without code modifications. Field autofill templates define patterns for pre-populating input values based on other field values, supporting both simple variable substitution (e.g., `{{ first_name }}.{{ last_name }}`) and complex string manipulation through Jinja filters (e.g., `{{ first_name[0]|upper }}{{ last_name }}`). External API endpoint patterns incorporate templating to construct context-specific URLs (e.g., `/api/customers/{{ idrs }}/adusers?domain={{ domain }}`), enabling reusable API configurations that adapt to different customers and field values.

The templating context includes all field names from the associated action, the customer's IDRS identifier, and customer code, providing comprehensive access to contextual information for template rendering. This templating capability is utilized by both backend validation logic and frontend presentation layer, ensuring consistent behavior across the system.

Target Machine Strategy Pattern

The determination of where service requests execute is abstracted through the Strategy pattern implemented in `ActionTargetTypes`. Each action specifies a `target_type` enumeration value (Domain Controller, Exchange Server, Internal Worker, or SRP Agent) that maps to a concrete strategy class implementing `BaseActionTargetType`. Each strategy encapsulates the logic for identifying the source field that determines the target (if applicable), validating its presence in the action's field set, extracting its value from service request fields, and resolving the final target machine hostname or IP address.

For example, the `DCFromDomainTargetType` strategy identifies the domain field (ID 112), validates it references customer parameters with group "DOMINIO", extracts the domain value from the service request, and resolves it to the corresponding Domain Controller FQDN through the `convert_domain_name_to_dc` utility. This abstraction enables addition of new target types without modifying core execution logic, simply by implementing the strategy interface and adding the corresponding enumeration value.

Additional Script Parameters Strategy Pattern

Beyond form field values, actions can specify additional parameter groups to be injected into execution scripts through the `additional_script_parameters_groups` array field. Available groups include Office 365 credentials, Domain Controller server derived from the domain field, and Exchange Server derived from the domain field. Each group is implemented as a strategy class inheriting from `BaseScriptParameterGroup`, which provides a `get_parameters_string` method that accepts a service request and returns the formatted parameter string to append to the script invocation.

The `DCServerFromSRDomainScriptParametersGroup` strategy, for instance, extracts the domain value from the service request, resolves it to the DC hostname, and returns a formatted parameter string like `-Server dc01.elmec.ad`. Service request execution iterates through the configured parameter groups, invokes each strategy, and concatenates the resulting parameter strings, transparently extending the script invocation with context-specific values without requiring script template modifications.

This catalog architecture achieves the primary objective of centralizing all configuration logic on the backend while providing administrators with powerful, intuitive customization capabilities that eliminate code modifications for common configuration scenarios. The three-level hierarchy, combined with the templating system and strategy patterns, delivers the flexibility required to support diverse customer requirements while maintaining a single, maintainable codebase.

3.3.4 Fields Validation

3.3.5 Message Queue System for Asynchronous Execution Flow

3.3.6 Implementation of Customers' Master Data Import

3.3.7 Logging and Monitoring

3.4 Frontend Application Implementation

3.4.1 Technological Overview

3.4.2 Service Request Creation Platform for End Users

3.4.3 Administrative Configuration Platform for System Administrators

Chapter 4

Migrations

- Planning and execution of a full migration of databases and other company services / platforms that use SRP;
- Strategies to minimize downtime and ensure data integrity during the migration process
- Strategies to ensure backward compatibility during the transition period
- Strategies to ensure a smooth co-existence of the two systems during the transition period

Chapter 5

Reccomendation system through RAG and Fine-tuned LLMs

- Integration of Retrieval-Augmented Generation (RAG) techniques
- Fine-tuning of Large Language Models (LLMs)
- Implementation of a recommendation system for service requests

Conclusions

- Reflections on the internship experience and acquired skills
- Considerations on the future of the project and potential developments

Bibliography

[1] *Active Directory Organizational Unit (OU).* URL: https://blog.netwrix.com/2024/02/19/active-directory-organizational-unit-ou/ (cit. on p. 23).

[2] Mashel Albarak and Rami Bahsoon. *Prioritizing Technical Debt in Database Normalization Using Portfolio Theory and Data Quality Metrics.* 2018. arXiv: 1801.06989 [cs.SE]. URL: https://arxiv.org/abs/1801.06989 (cit. on p. 16).

[3] Charles Anderson. "Docker [Software engineering]". In: *IEEE Software* 32.3 (2015), pp. 102–c3. DOI: 10.1109/MS.2015.62 (cit. on p. 28).

[4] *Ansible Documentation.* URL: https://docs.ansible.com/ansible/latest/dev_guide/overview_architecture.html (cit. on pp. 33, 34).

[5] *Automation with Cron job on Centos 8.* URL: https://comtronic.com.au/automation-with-cron-job-on-centos-8/ (cit. on p. 53).

[6] *Basics of Django: Model-View-Template (MVT) Architecture.* URL: https://angelogentileiii.medium.com/basics-of-django-model-view-template-mvt-architecture-8585aecffbf6 (cit. on p. 55).

[7] Michael R. Blaha. "Referential Integrity Is Important For Databases". In: 2005. URL: https://api.semanticscholar.org/CorpusID:6831872 (cit. on p. 16).

[8] *Celery - Distributed Task Queue.* URL: https://docs.celeryq.dev/en/stable/getting-started/introduction.html (cit. on p. 58).

[9] *Cryptsetup and LUKS - open-source disk encryption.* URL: https://gitlab.com/cryptsetup/cryptsetup (cit. on p. 11).

[10] Lorenzo De Lauretis. "From Monolithic Architecture to Microservices Architecture". In: *2019 IEEE International Symposium on Software Reliability Engineering Workshops (ISSREW).* 2019, pp. 93–96. DOI: 10.1109/ISSREW.2019.00050 (cit. on p. 24).

[11] *Django - The web framework for perfectionists with deadlines.* URL: https://www.djangoproject.com/ (cit. on pp. 55, 57, 65).

[12] *Django REST framework.* URL: https://www.django-rest-framework.org/ (cit. on pp. 55–57).

[13] *DMI Infrastructure.* Internal repository, not publicly accessible. URL: https://git.elmec.com/publicautomation/dmi-infrastructure.git (cit. on pp. 8, 10, 11).

[14] Roy T. Fielding. *Architectural Styles and the Design of Network-Based Software Architectures.* University of California, Irvine, 2000 (cit. on p. 26).

[15] *GORM - The fantastic ORM library for Golang.* URL: https://gorm.io/docs/ (cit. on p. 56).

[16] Paul J. Deitel Harvey M. Deitel. *C++. Fondamenti di programmazione*. Apogeo, 2005 (cit. on p. 16).

[17] *HDA - Help Desk Advanced*. URL: https://pat.eu/en/products/helpdeskadvanced/ (cit. on p. 27).

[18] *K3s - Lightweight Kubernetes*. URL: https://docs.k3s.io/ (cit. on p. 11).

[19] *Kubernetes: Production-Grade Container Orchestration*. URL: https://kubernetes.io/docs/concepts/overview/ (cit. on pp. 27, 29).

[20] *Mastering Celery: A Guide to Background Tasks, Workers, and Parallel Processing in Python*. URL: https://khairi-brahmi.medium.com/mastering-celery-a-guide-to-background-tasks-workers-and-parallel-processing-in-python-eea575928c52 (cit. on p. 58).

[21] Severi Peltonen, Luca Mezzalira, and Davide Taibi. *Motivations, Benefits, and Issues for Adopting Micro-Frontends: A Multivocal Literature Review*. 2021. arXiv: 2007.00293 [cs.SE]. URL: https://arxiv.org/abs/2007.00293 (cit. on pp. 12, 24).

[22] *Pod Quality of Service Classes*. URL: https://kubernetes.io/docs/concepts/workloads/pods/pod-qos/ (cit. on p. 50).

[23] *Provision Prime*. Internal repository, not publicly accessible. URL: https://git.elmec.com/innovation/development/provisionprime.git (cit. on pp. 23, 24, 27).

[24] *Refactoring Guru - Strategy Pattern*. URL: https://refactoring.guru/design-patterns/strategy (cit. on p. 34).

[25] David K. Rensin. *Kubernetes - Scheduling the Future at Cloud Scale*. 2015, All. URL: http://www.oreilly.com/webops-perf/free/kubernetes.csp (cit. on pp. 29, 30).

[26] Elmec Informatica S.p.A. *Azioni Gestione SRP 4.0*. Internal document, not publicly available., 2016 (cit. on pp. 3, 5, 6, 26).

[27] Elmec Informatica S.p.A. *Requirements definition meeting for the new system*. Internal meeting. Held with the stakeholders and developers of the old system to gather requirements for the new system. Apr. 2025 (cit. on pp. 8, 16, 19–24).

[28] Elmec Informatica S.p.A. *Service Request Portal*. Internal document, not publicly available., 2016 (cit. on pp. 8, 9, 22).

[29] *Semgrep App Security Platform | AI-Assisted SAST, SCA and Secrets Detection*. URL: https://semgrep.dev/ (cit. on p. 41).

[30] *SQL Server Integration Services*. URL: https://learn.microsoft.com/en-us/sql/integration-services/sql-server-integration-services (cit. on p. 14).

[31] *SRP Internal Worker Windows Service*. Internal repository, not publicly accessible. URL: https://git.elmec.com/automationcloud/srpinternalworker_windowsservice.git (cit. on p. 10).

[32] *SRP Web Service*. Internal repository, not publicly accessible. URL: https://git.elmec.com/automationcloud/SRP_WebService.git (cit. on p. 9).

[33] *SRP Web Service REST*. Internal repository, not publicly accessible. URL: https://git.elmec.com/automationcloud/srp_webservicerest.git (cit. on pp. 9, 19, 22, 24).

[34] *SRP Windows Service*. Internal repository, not publicly accessible. URL: https://git.elmec.com/automationcloud/SRP_WindowsService.git (cit. on pp. 7, 10, 16, 22–24).

[35] Michael Stonebraker, Lawrence Rowe, and Michael Hirohama. "The Implementation Of Postgres". In: *Knowledge and Data Engineering, IEEE Transactions on* 2 (Apr. 1990), pp. 125–142. DOI: 10.1109/69.50912 (cit. on p. 35).

[36] *Stored Procedures (Database Engine)*. URL: https://learn.microsoft.com/en-us/sql/relational-databases/stored-procedures/stored-procedures-database-engine (cit. on p. 14).

[37] Jan Tretmans and Louis Verhaard. "A Queue Model Relating Synchronous and Asynchronous Communication". In: *Protocol Specification, Testing and Verification, XII*. Ed. by R.J. LINN and M.Ü. UYAR. IFIP Transactions C: Communication Systems. Amsterdam: Elsevier, 1992, pp. 131–145. ISBN: 978-0-444-89874-6. DOI: https://doi.org/10.1016/B978-0-444-89874-6.50015-5. URL: https://www.sciencedirect.com/science/article/pii/B9780444898746500155 (cit. on p. 27).

[38] *Trivy - The all-in-one open source security scanner*. URL: https://trivy.dev/ (cit. on p. 40).

[39] *TypeScript: JavaScript with Syntax for Types*. URL: https://www.typescriptlang.org/docs/handbook/typescript-from-scratch.html (cit. on p. 30).

[40] Brecht Van de Vyvere, Pieter Colpaert, and Ruben Verborgh. "Comparing a Polling and Push-Based Approach for Live Open Data Interfaces". In: *Web Engineering*. Ed. by Maria Bielikova, Tommi Mikkonen, and Cesare Pautasso. Cham: Springer International Publishing, 2020, pp. 87–101. ISBN: 978-3-030-50578-3 (cit. on p. 22).

[41] *Vault - Manage Secrets and Protect Sensitive Data*. URL: https://developer.hashicorp.com/vault (cit. on p. 50).

[42] *Vue.js v3 Documentation*. URL: https://vuejs.org/guide/introduction (cit. on pp. 30, 31).

[43] *What is a Container?* URL: https://www.docker.com/resources/what-container/ (cit. on p. 29).

[44] *What is referential integrity and what does it look like in practice?* URL: https://www.y42.com/blog/referential-integrity (cit. on p. 16).